



Project 1: **37** days left

Offense-Based Cyber Security: Exploitation of Memory Vulnerabilities

CS 459/559: Science of Cyber Security
5th Lecture

Instructor:

Guanhua Yan

Project 1

- Demonstrate a cyber attack
- In your first project report, please:
 - Explain what are the vulnerability, exploit, attack surface, and attack vector in your project
 - Explain how the attack (exploitation) occurs in your project
 - Explain why the attack (exploitation) works
- Each of your project reports should be **at least five pages, excluding bibliography**
- **Due time: October 8, 2026**
 - **Four days of grace period, with 2.5% penalty each half day late**
- **Grading criteria:**
 - **Results, novelty, difficulty, presentation**

How to choose your project?

- You should be familiar with the concepts behind the attack (software, network, system, or human)
- You should be comfortable with developing defensive mechanisms on the target being attacked
- It's OK to work on known vulnerabilities using known exploit code (plenty of online resources)
 - Known vulnerabilities: National Vulnerability Database (<https://nvd.nist.gov>)
 - Known exploits: Exploit database (<https://exploit-db.com>)
 - The metasploit framework (<https://www.metasploit.com>) makes exploitation easy!
- Start early and work on your project hard!
 - **You will continue working on the same project with defensive techniques introduced in this course**

Agenda

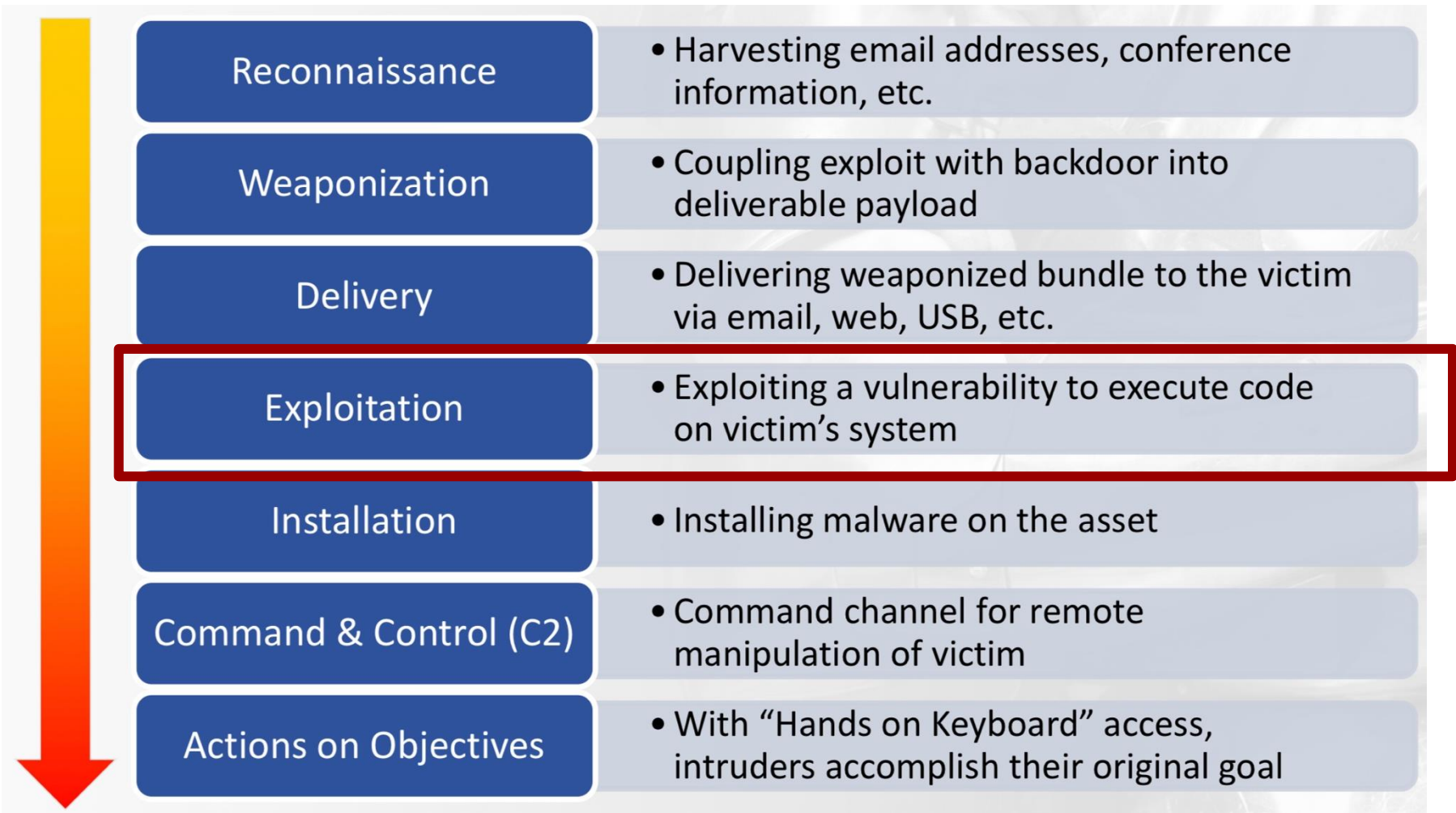
- **Sept 1: memory-level attack**
- **Sept 3: web-level attack**
- **Sept 8: no class**
- **Sept 10: system-level attack**
- **Sept 15: hardware-level attack**
- **Sept 17: human-level attack**
- **Sept 22: Quiz 1**
- **October 8: Project 1 attack report due**
- **October 12-16: Attack demo**

What we have learned from last lecture

■ Network vulnerability exploitation

- Cache poisoning attacks
- Sniffing
- Denial of service attacks

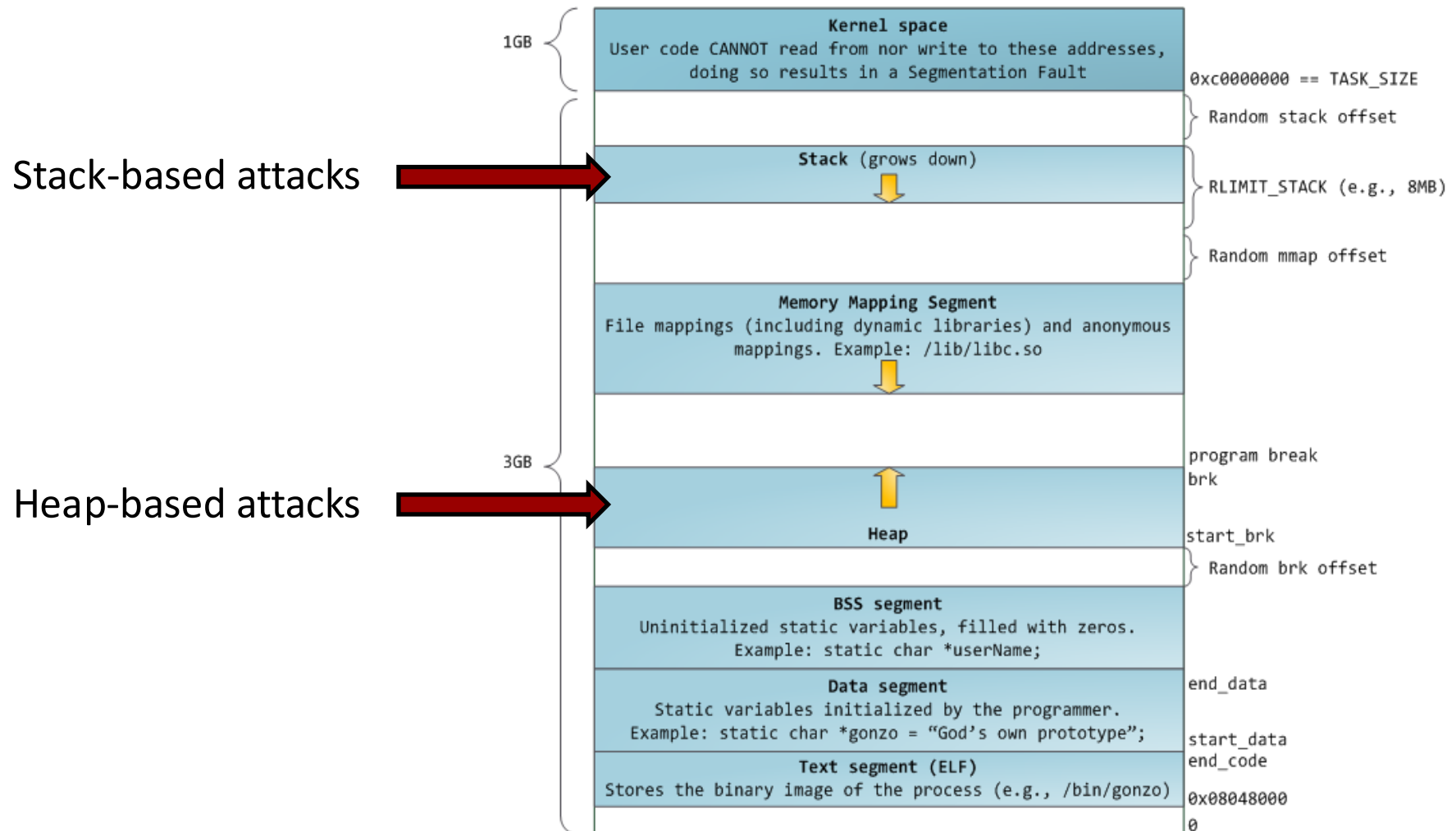
Exploitation of memory vulnerabilities



What we are going to learn

- *Stack-based attacks*
- *Heap-based attacks*

Run-time Linux process memory layout



Stack-based attacks

Example vulnerable C code

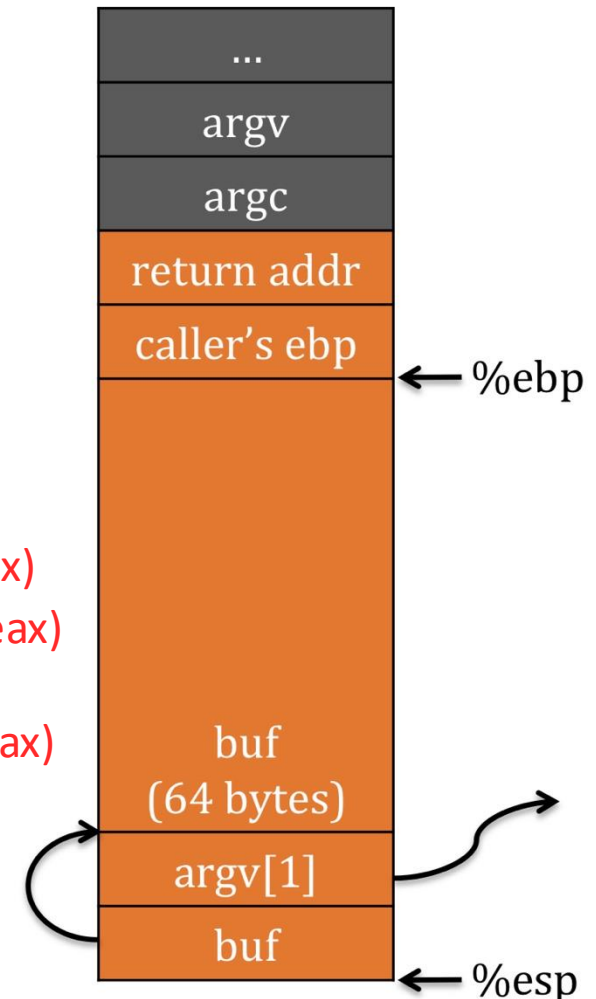
```
leave:
mov %ebp, %esp
pop %ebp
```

```
#include<string.h>
int main(int argc, char **argv) {
    char buf[64];
    strcpy(buf, argv[1]);
}
```

Dump of assembler code for function main:

```
0x080483e4 <+0>: push    %ebp
0x080483e5 <+1>: mov     %esp,%ebp
0x080483e7 <+3>: sub     $72,%esp
0x080483ea <+6>: mov     12(%ebp),%eax (argv→eax)
0x080483ed <+9>: mov     4(%eax),%eax (argv[1]→eax)
0x080483f0 <+12>: mov     %eax,4(%esp)
0x080483f4 <+16>: lea     -64(%ebp),%eax (buf→eax)
0x080483f7 <+19>: mov     %eax,(%esp)
0x080483fa <+22>: call    0x8048300 <strcpy@plt>
0x080483ff <+27>: leave   (mov ebp, esp; pop ebp)
0x08048400 <+28>: ret
```

Higher Address



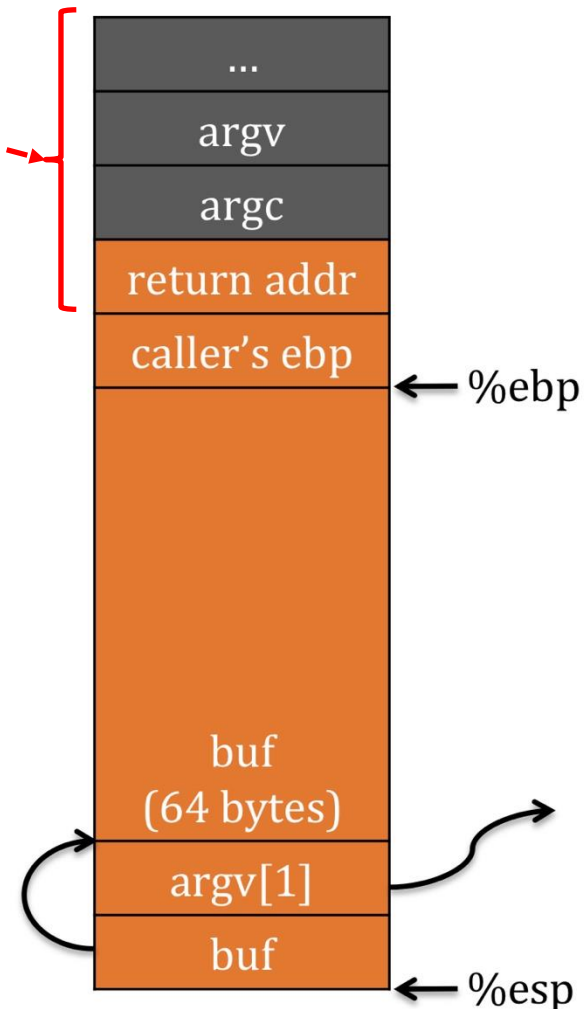
Example vulnerable C code

```
#include<string.h>
int main(int argc, char **argv) {
    char buf[64];
    strcpy(buf, argv[1]);
}
```

Dump of assembler code for function main:

```
0x080483e4 <+0>: push    %ebp
0x080483e5 <+1>: mov     %esp,%ebp
0x080483e7 <+3>: sub     $72,%esp
0x080483ea <+6>: mov     12(%ebp),%eax
0x080483ed <+9>: mov     4(%eax),%eax
0x080483f0 <+12>: mov     %eax,4(%esp)
0x080483f4 <+16>: lea     -64(%ebp),%eax
0x080483f7 <+19>: mov     %eax,(%esp)
0x080483fa <+22>: call    0x8048300 <strcpy@plt>
0x080483ff <+27>: leave
0x08048400 <+28>: ret
```

Higher Address



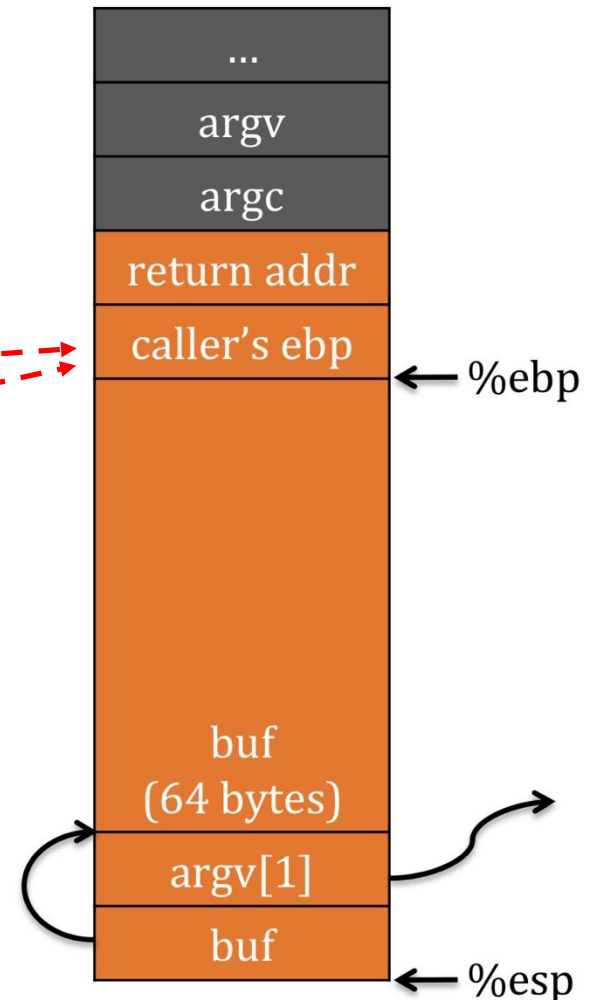
Example vulnerable C code

```
#include<string.h>
int main(int argc, char **argv) {
    char buf[64];
    strcpy(buf, argv[1]);
}
```

Dump of assembler code for function main:

```
0x080483e4 <+0>: push    %ebp
0x080483e5 <+1>: mov     %esp,%ebp
0x080483e7 <+3>: sub     $72,%esp
0x080483ea <+6>: mov     12(%ebp),%eax
0x080483ed <+9>: mov     4(%eax),%eax
0x080483f0 <+12>: mov     %eax,4(%esp)
0x080483f4 <+16>: lea     -64(%ebp),%eax
0x080483f7 <+19>: mov     %eax,(%esp)
0x080483fa <+22>: call    0x8048300 <strcpy@plt>
0x080483ff <+27>: leave
0x08048400 <+28>: ret
```

Higher Address



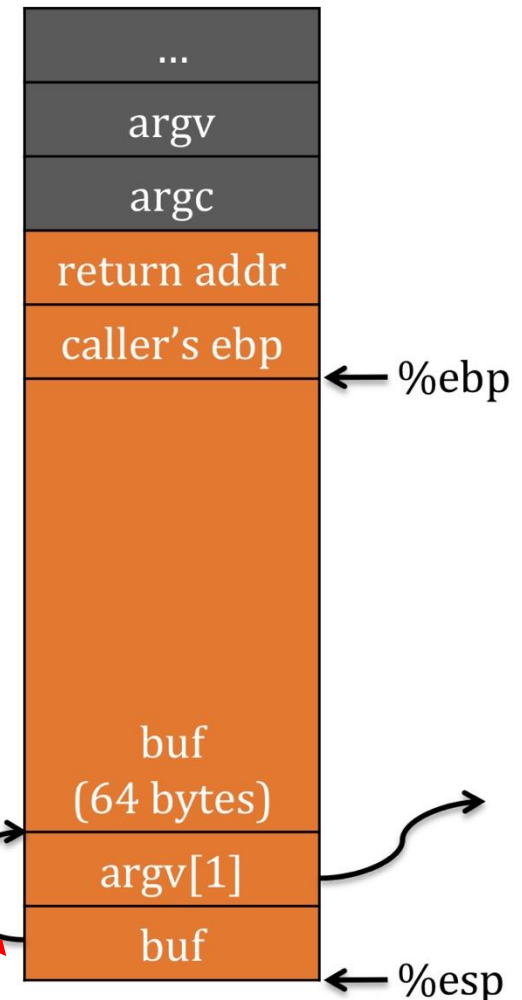
Example vulnerable C code

```
#include<string.h>
int main(int argc, char **argv) {
    char buf[64];
    strcpy(buf, argv[1]);
}
```

Dump of assembler code for function main:

```
0x080483e4 <+0>: push    %ebp
0x080483e5 <+1>: mov     %esp,%ebp
0x080483e7 <+3>: sub     $72,%esp
0x080483ea <+6>: mov     12(%ebp),%eax
0x080483ed <+9>: mov     4(%eax),%eax
0x080483f0 <+12>: mov     %eax,4(%esp)
0x080483f4 <+16>: lea     -64(%ebp),%eax
0x080483f7 <+19>: mov     %eax,(%esp)
0x080483fa <+22>: call    0x8048300 <strcpy@plt>
0x080483ff <+27>: leave
0x08048400 <+28>: ret
```

Higher Address



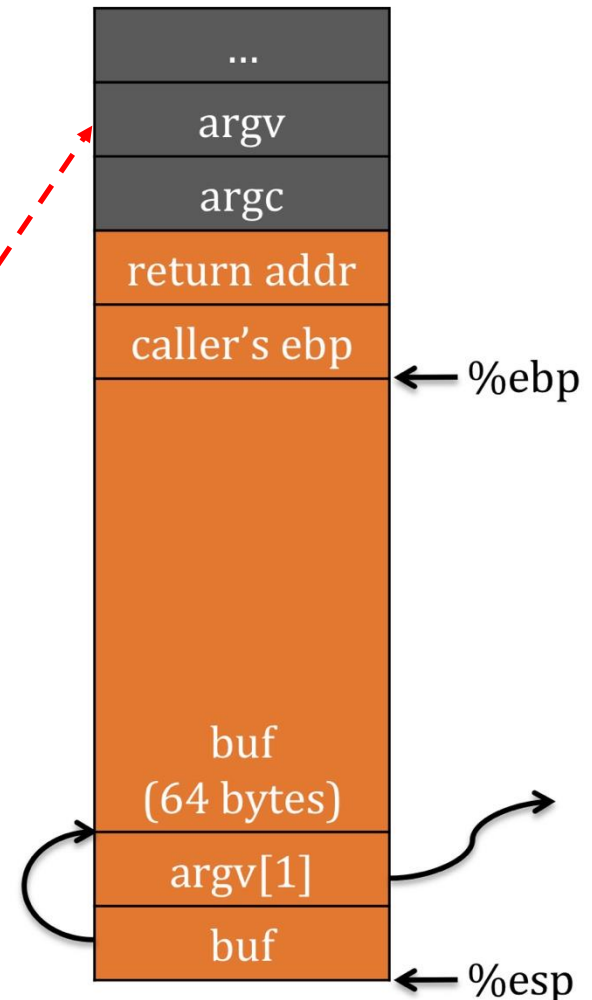
Example vulnerable C code

```
#include<string.h>
int main(int argc, char **argv) {
    char buf[64];
    strcpy(buf, argv[1]);
}
```

Dump of assembler code for function main:

```
0x080483e4 <+0>: push    %ebp
0x080483e5 <+1>: mov     %esp,%ebp
0x080483e7 <+3>: sub     $72,%esp
0x080483ea <+6>: mov     12(%ebp),%eax
0x080483ed <+9>: mov     4(%eax),%eax
0x080483f0 <+12>: mov     %eax,4(%esp)
0x080483f4 <+16>: lea     -64(%ebp),%eax
0x080483f7 <+19>: mov     %eax,(%esp)
0x080483fa <+22>: call    0x8048300 <strcpy@plt>
0x080483ff <+27>: leave
0x08048400 <+28>: ret
```

Higher Address



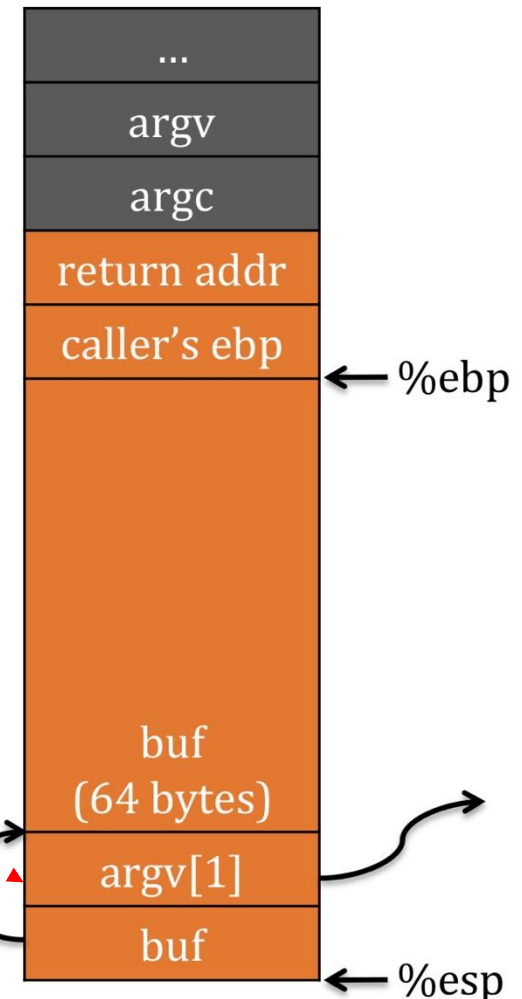
Example vulnerable C code

```
#include<string.h>
int main(int argc, char **argv) {
    char buf[64];
    strcpy(buf, argv[1]);
}
```

Dump of assembler code for function main:

```
0x080483e4 <+0>: push    %ebp
0x080483e5 <+1>: mov     %esp,%ebp
0x080483e7 <+3>: sub     $72,%esp
0x080483ea <+6>: mov     12(%ebp),%eax
0x080483ed <+9>: mov     4(%eax),%eax
0x080483f0 <+12>: mov     %eax,4(%esp)
0x080483f4 <+16>: lea     -64(%ebp),%eax
0x080483f7 <+19>: mov     %eax,(%esp)
0x080483fa <+22>: call    0x8048300 <strcpy@plt>
0x080483ff <+27>: leave
0x08048400 <+28>: ret
```

Higher Address



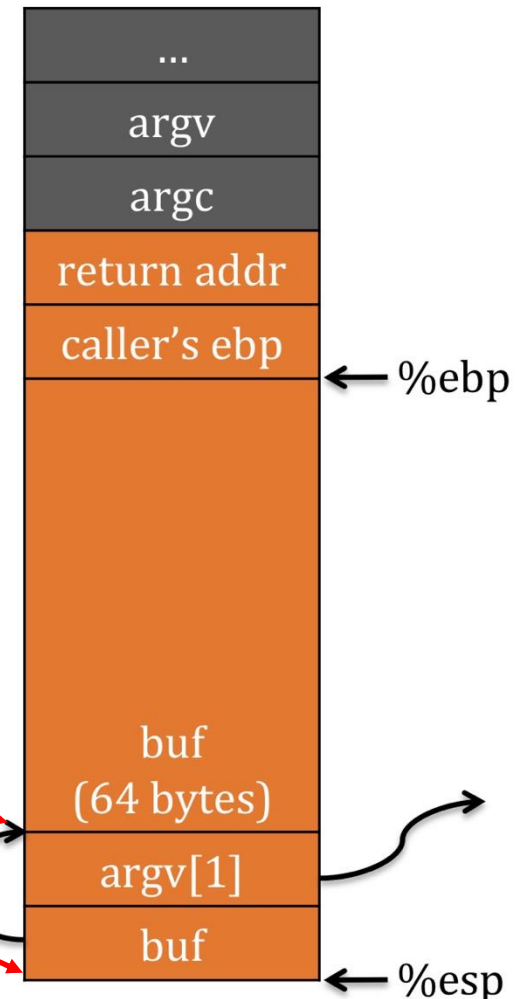
Example vulnerable C code

```
#include<string.h>
int main(int argc, char **argv) {
    char buf[64];
    strcpy(buf, argv[1]);
}
```

Dump of assembler code for function main:

```
0x080483e4 <+0>: push    %ebp
0x080483e5 <+1>: mov     %esp,%ebp
0x080483e7 <+3>: sub     $72,%esp
0x080483ea <+6>: mov     12(%ebp),%eax
0x080483ed <+9>: mov     4(%eax),%eax
0x080483f0 <+12>: mov     %eax,4(%esp)
0x080483f4 <+16>: lea     -64(%ebp),%eax
0x080483f7 <+19>: mov     %eax,(%esp)
0x080483fa <+22>: call    0x8048300 <strcpy@plt>
0x080483ff <+27>: leave
0x08048400 <+28>: ret
```

Higher Address

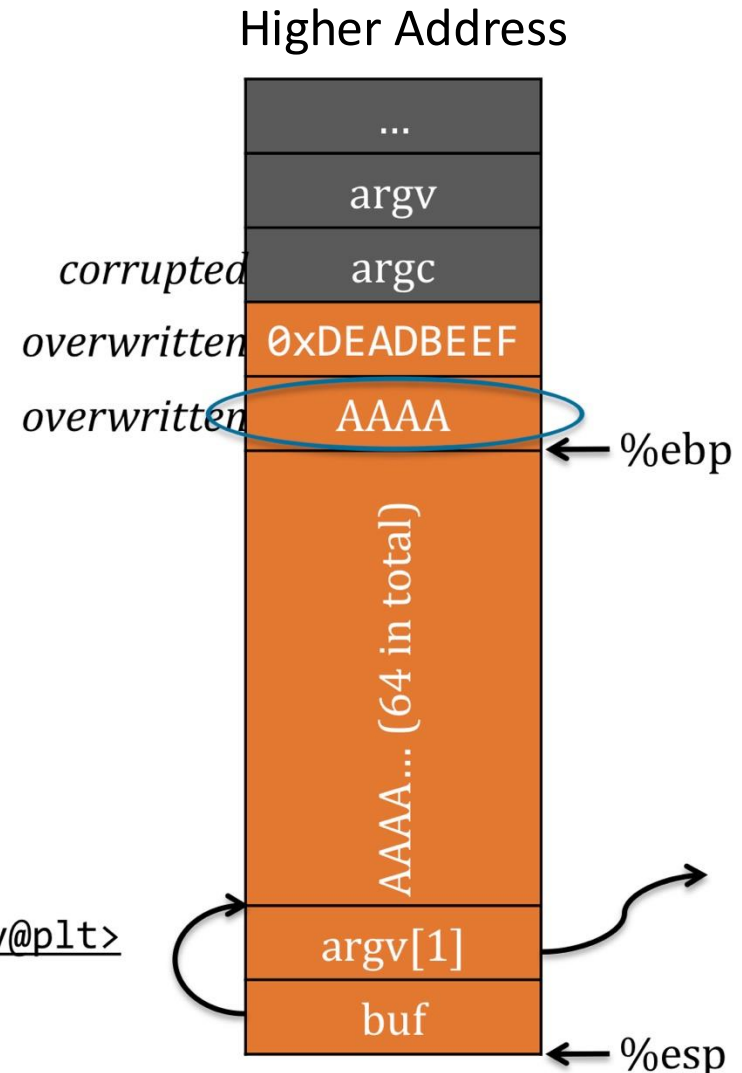


Buffer overflow attack

```
#include<string.h>
int main(int argc, char **argv) {
    char buf[64];
    strcpy(buf, argv[1]);
}
```

Dump of assembler code for function main:

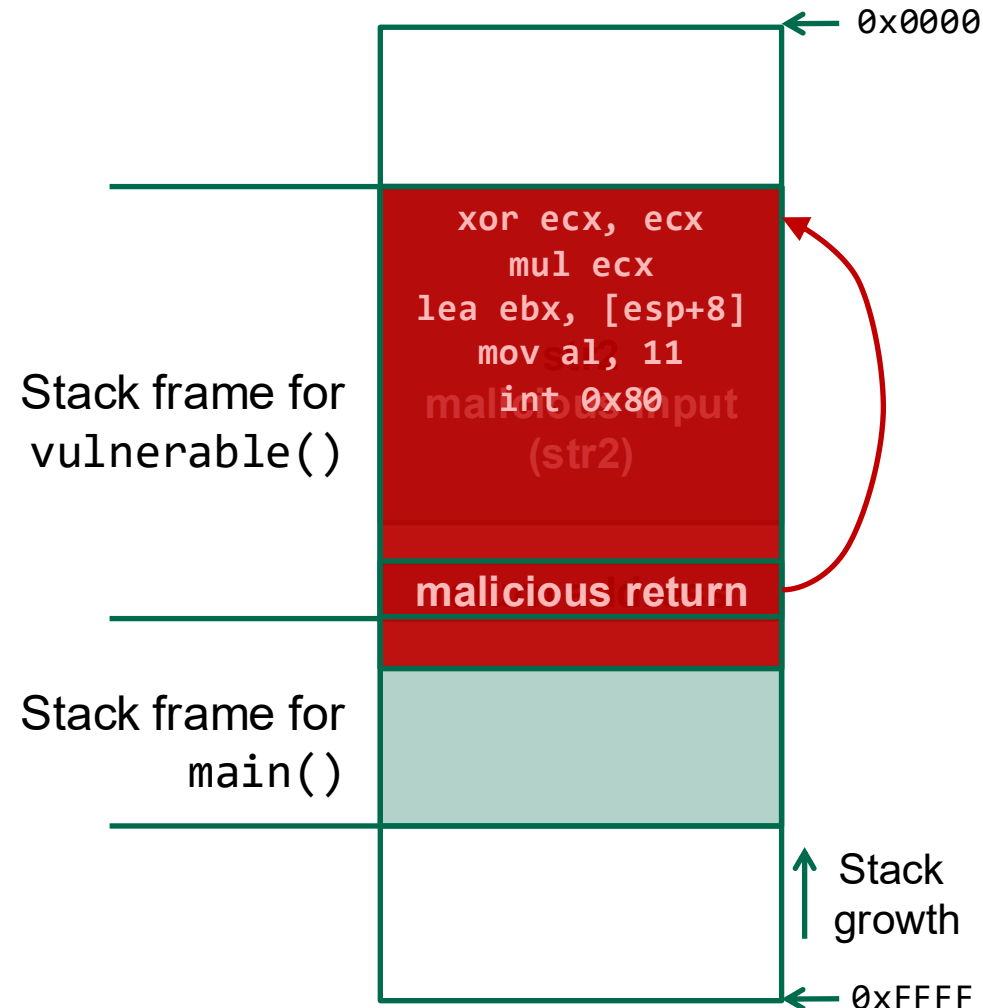
```
0x080483e4 <+0>: push    %ebp
0x080483e5 <+1>: mov     %esp,%ebp
0x080483e7 <+3>: sub     $72,%esp
0x080483ea <+6>: mov     12(%ebp),%eax
0x080483ed <+9>: mov     4(%eax),%eax
0x080483f0 <+12>: mov     %eax,4(%esp)
0x080483f4 <+16>: lea     -64(%ebp),%eax
0x080483f7 <+19>: mov     %eax,(%esp)
0x080483fa <+22>: call    0x8048300 <strcpy@plt>
0x080483ff <+27>: leave
0x08048400 <+28>: ret
```



Buffer Overflow and Code Injection Attack: Example

```
→ main (int argc, char **argv)
{
    ...
→ vulnerable(argv[1]);
    ...
}
```

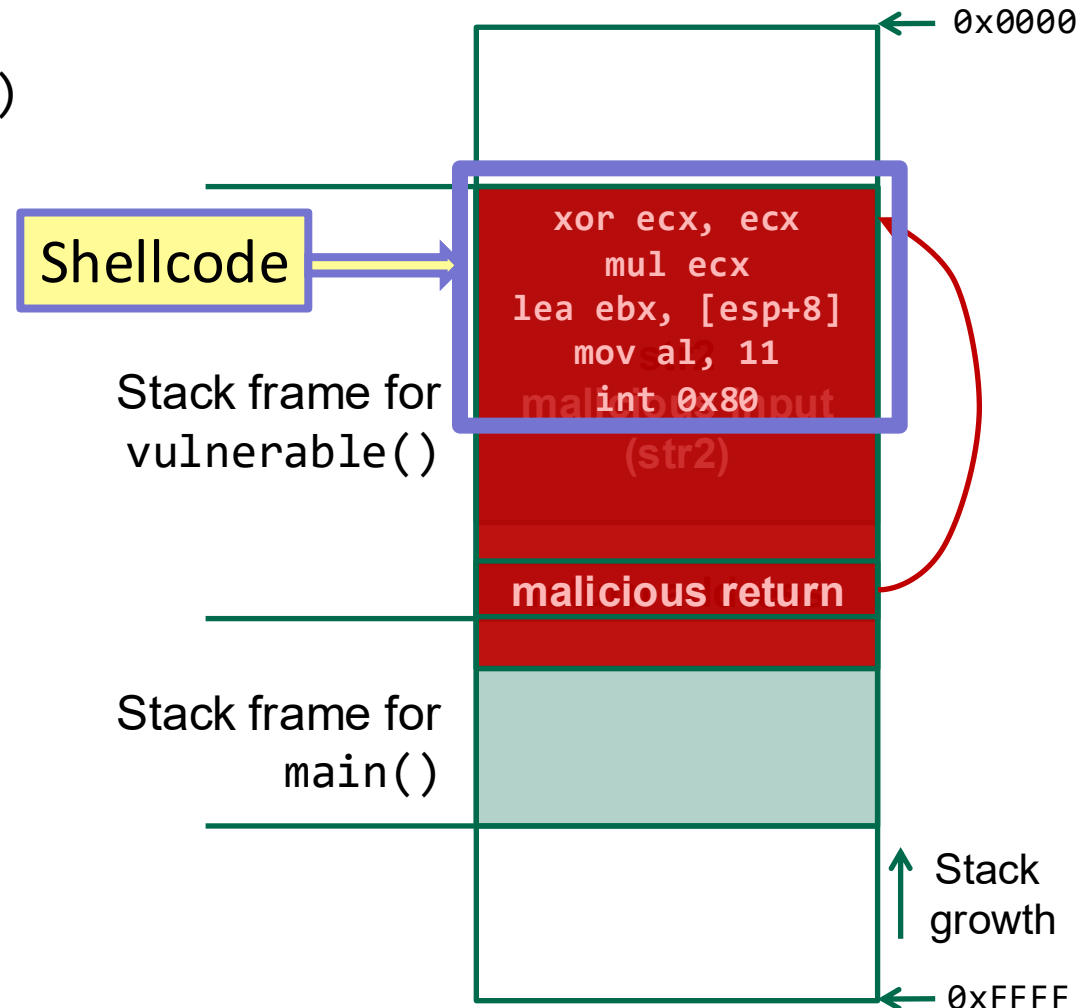
```
→ vulnerable(char *str1)
{
→ char str2[100];
→ strcpy(str2, str1);
→ return;
}
```



Shellcode: a piece of code used as payload when exploiting a software vulnerability

```
→ main (int argc, char **argv)
{
    ...
→ vulnerable(argv[1]);
    ...
}
```

```
→ vulnerable(char *str1)
{
→ char str2[100];
→ strcpy(str2, str1);
→ return;
}
```



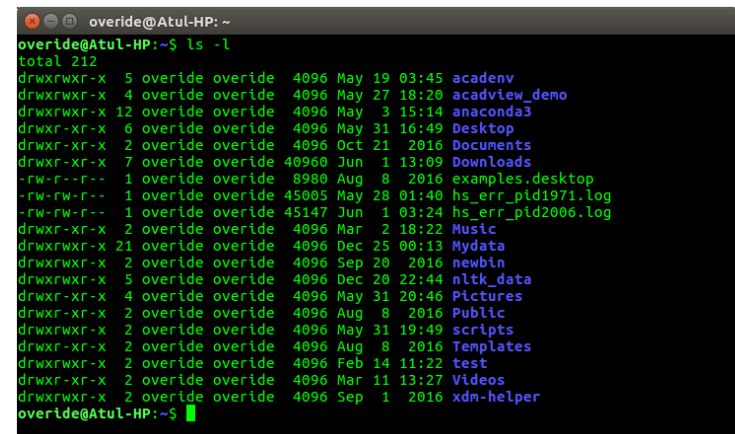
How to generate shellcode?

- Spawn a shell: a program that takes commands from the keyboard and gives them to the OS to perform.

```
#include <unistd.h>

int main() {
    char *args[2];
    args[0] = "/bin/sh";
    args[1] = NULL;
    execve(args[0], args, NULL);
}
```

A new shell!



```
override@Atul-HP: ~
override@Atul-HP:~$ ls -l
total 212
drwxrwxr-x 5 override override 4096 May 19 03:45 acadenv
drwxrwxr-x 4 override override 4096 May 27 18:28 acadview_demo
drwxrwxr-x 12 override override 4096 May 3 15:14 anaconda3
drwxr-xr-x 6 override override 4096 May 31 16:49 Desktop
drwxr-xr-x 2 override override 4096 Oct 21 2016 Documents
drwxr-xr-x 7 override override 40960 Jun 1 13:09 Downloads
-rw-r--r-- 1 override override 8980 Aug 8 2016 examples.desktop
-rw-rw-r-- 1 override override 45005 May 28 01:40 hs_err_pid1971.log
-rw-rw-r-- 1 override override 45147 Jun 1 03:24 hs_err_pid2006.log
drwxr-xr-x 2 override override 4096 Mar 2 18:22 Music
drwxrwxr-x 21 override override 4096 Dec 25 00:13 Mydata
drwxrwxr-x 2 override override 4096 Sep 20 2016 newbin
drwxrwxr-x 5 override override 4096 Dec 20 22:44 nltk_data
drwxr-xr-x 4 override override 4096 May 31 20:46 Pictures
drwxr-xr-x 2 override override 4096 Aug 8 2016 Public
drwxrwxr-x 2 override override 4096 May 31 19:49 scripts
drwxr-xr-x 2 override override 4096 Aug 8 2016 Templates
drwxrwxr-x 2 override override 4096 Feb 14 11:22 test
drwxr-xr-x 2 override override 4096 Mar 11 13:27 Videos
drwxrwxr-x 2 override override 4096 Sep 1 2016 xdm-helper
override@Atul-HP:~$
```

ssh

bash

```
ghyan@bravo:~/teaching$ ls
a.out get_shell get_shell2.c get_shell.asm get_shell.c get_shell.o x2.c x.c y.c
ghyan@bravo:~/teaching$ gcc ./get_shell.c -o ./get_shell
ghyan@bravo:~/teaching$ ./get_shell
$
```

Assembly to binary shellcode

get_shell.asm

```

jmp short    mycall          ; Immediately jump to the call instruction

shellcode:
    pop      esi              ; Store the address of "/bin/sh" in ESI
    xor      eax, eax         ; Zero out EAX
    mov byte [esi + 7], al    ; Write the null byte at the end of the string

    mov dword [esi + 8], esi  ; [ESI+8], i.e. the memory immediately below the string
                                ; "/bin/sh", will contain the array pointed to by the
                                ; second argument of execve(2); therefore we store in
                                ; [ESI+8] the address of the string...
    mov dword [esi + 12], eax ; ...and in [ESI+12] the NULL pointer (EAX is 0)
    mov      al, 0xb          ; Store the number of the syscall (11) in EAX
    lea      ebx, [esi]        ; Copy the address of the string in EBX
    lea      ecx, [esi + 8]    ; Second argument to execve(2)
    lea      edx, [esi + 12]   ; Third argument to execve(2) (NULL pointer)
    int      0x80             ; Execute the system call

mycall:
    call     shellcode        ; Push the address of "/bin/sh" onto the stack
    db       "/bin/sh"

```



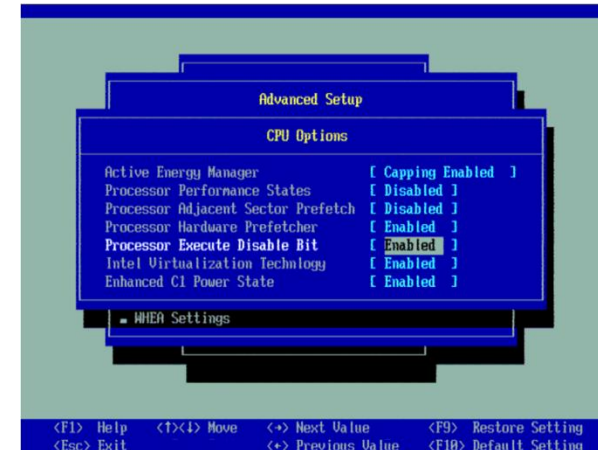
```

char shellcode[] = "\xeb\x18\x5e\x31\xc0\x88\x46\x07\x89\x76\x08\x89\x46"
                  "\x0c\xb0\x0b\x8d\x1e\x8d\x4e\x08\x8d\x56\x0c\xcd\x80"
                  "\xe8\xe3\xff\xff\xff\x2f\x62\x69\x6e\x2f\x73\x68";

```

Code Reuse Attacks

- Key Idea: Reuse existing library code instead of code injection
- Bypass NX (Non-Executable)
- Return-to-libc attacks
- Return Oriented Programming
- **Jump Oriented Programming**



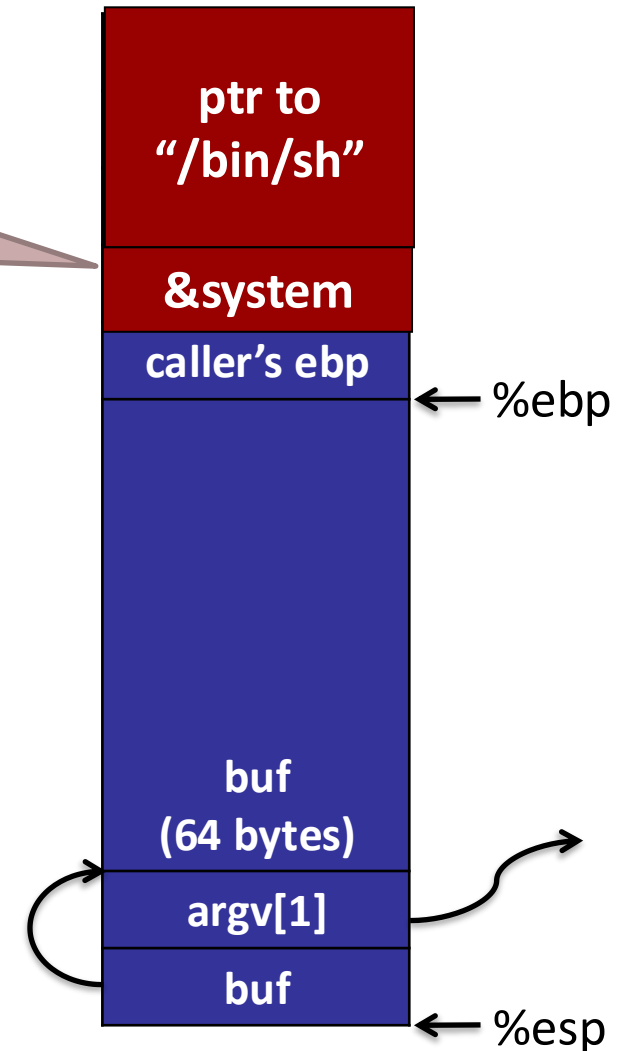
Return-to-libc attack

ret transfers control to system, which finds arguments on stack

Overwrite return address with address of libc function

- setup fake return address and argument(s)
- ret will “call” libc function

No injected code!



Return Oriented Programming Attacks

■ Turing-complete

- X86
- SPARC
- ARM

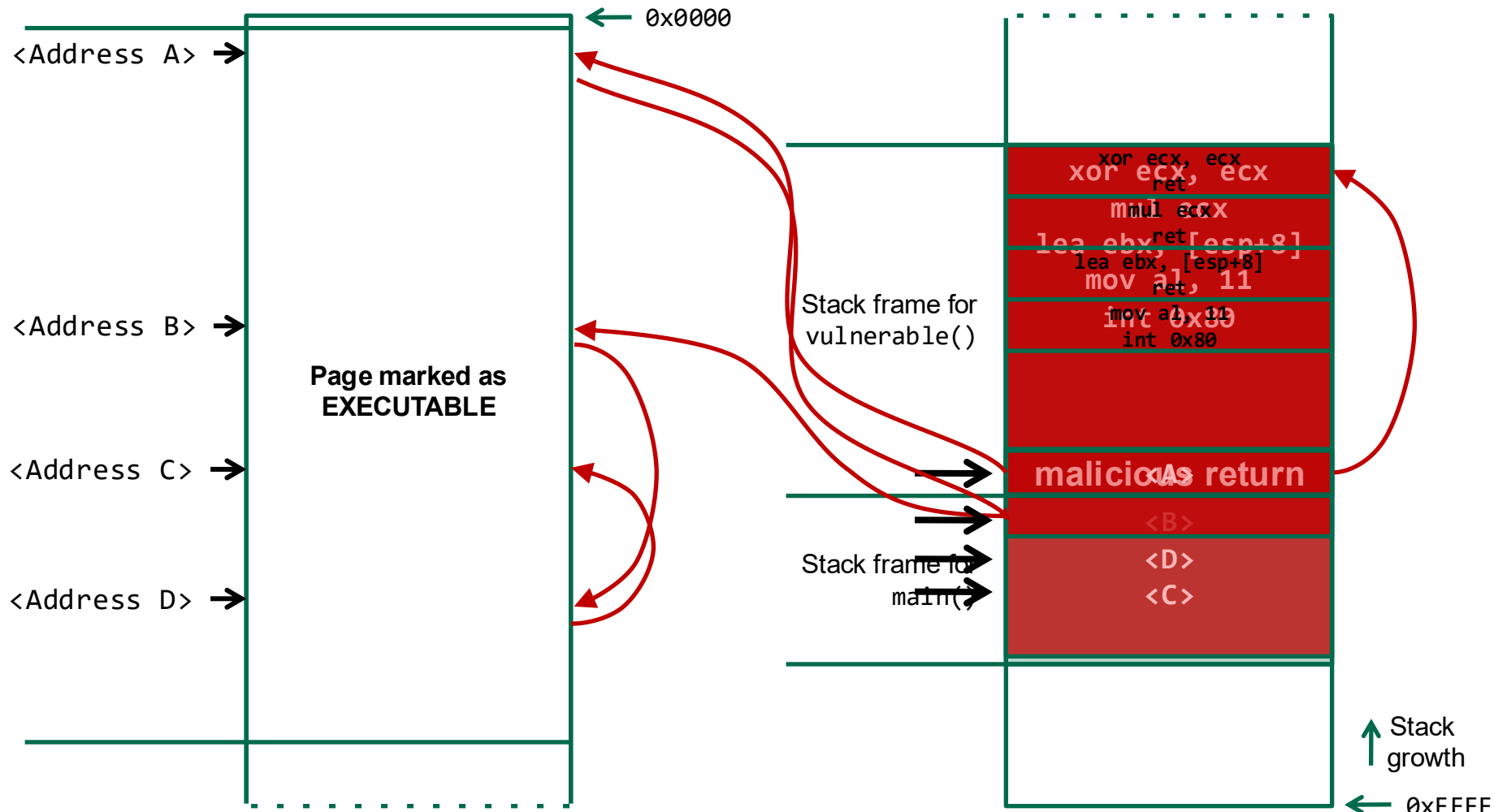
■ Exploits

- Voting machine
- Atmel sensor
- Cisco router
- Xen hypervisor
- Jailbreak
- Pwn2Own

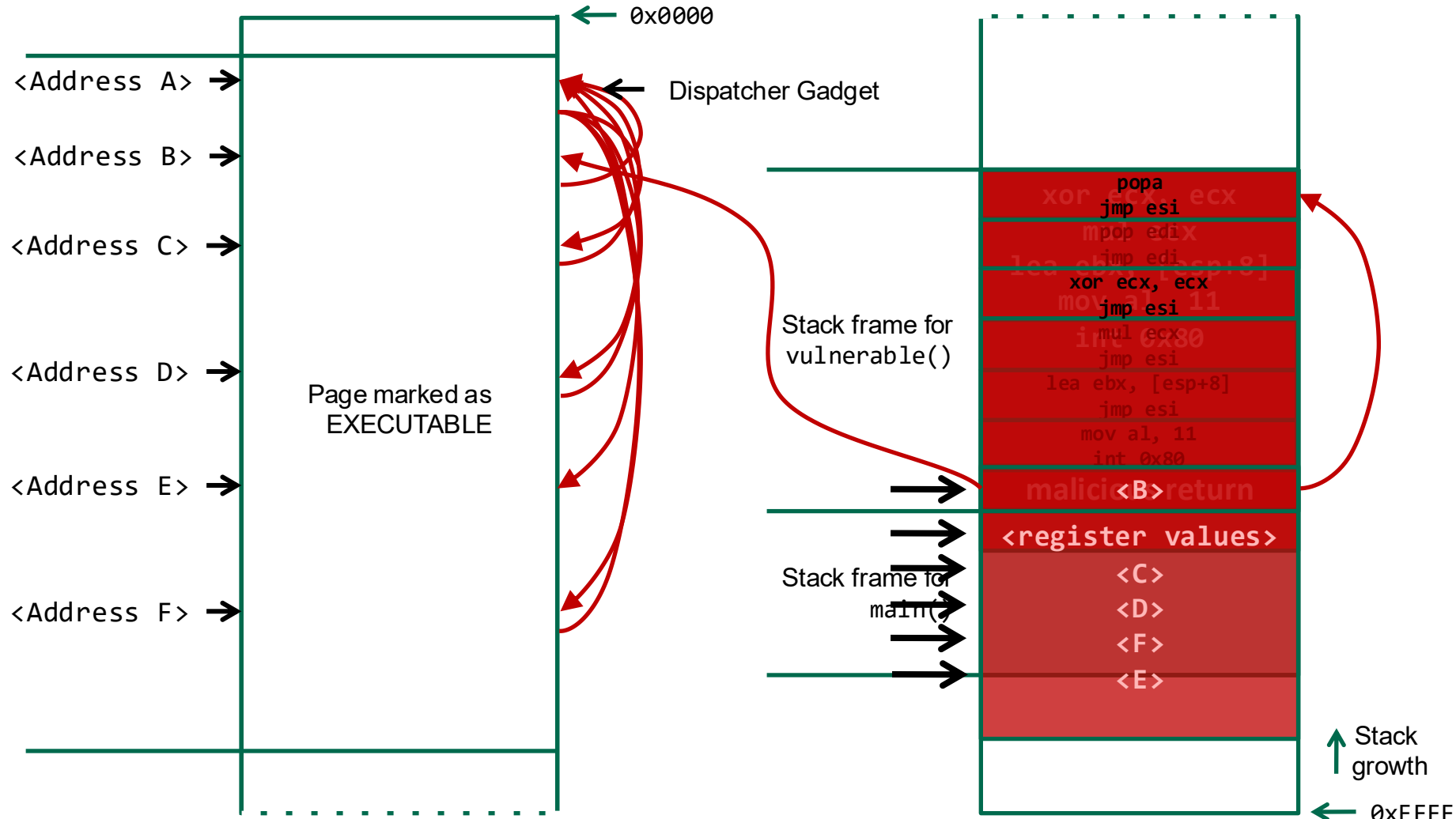
■ Automated tools

■ Microsoft BlueHat Prize (\$260K)

Return-Oriented Programming (ROP)



Jump Oriented Programming



pusha/popa

pusha

This instruction pushes all the general purpose registers onto the stack in the following order: AX, CX, DX, BX, SP, BP, SI, DI. The value of SP pushed is the value before the instruction is executed. It is useful for saving state before an operation that could potentially change these registers.

popa

This instruction pops all the general purpose registers off the stack in the reverse order of PUSH. That is, DI, SI, BP, SP, BX, DX, CX, AX. Used to restore state after a call to PUSH.

Heap-based attacks

The heap

- The heap is a pool of memory used for dynamic allocations at runtime
 - `malloc()` grabs memory on the heap
 - `free()` releases memory on the heap

Basics of Dynamic Memory

```
int main()
{
    char * buffer = NULL;

    /* allocate a 0x100 byte buffer */
    buffer = malloc(0x100);

    /* read input and print it */
    fgets(stdin, buffer, 0x100);
    printf("Hello %s!\n", buffer);

    /* destroy our dynamically allocated buffer */
    free(buffer);
    return 0;
}
```

```

push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
mov     short loc_313066, eax
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jz      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
push    eax
push    edi
mov     [ebp+arg_0], eax
call    sub_31486A
test    eax, eax
jz      short loc_31306D
push    esi
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 1D0h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31306F

loc_313066:                                     ; CODE XREF: sub_312FD8
                                              ; sub_312FD8+56
push    0Dh
call    sub_31411B

loc_31306D:                                     ; CODE XREF: sub_312FD8
                                              ; sub_312FD8+49
call    sub_3140F3
test    eax, eax
jg      short loc_31307D
call    sub_3140F3
jmp     short loc_31308C

; -----
loc_31307D:                                     ; CODE XREF: sub_312FD8
                                              ; sub_312FD8+5E
mov     [ebp+var_41], eax

```

Heap vs. Stack

■ Heap

- Dynamic memory allocations at runtime
- Objects, big buffers, structs, persistence, larger things
- Slower, manual
 - Done by the programmer
 - malloc/calloc/realloc/free
 - new/delete

■ Stack

- Fixed memory allocations known at compile time
- Local variables, return addresses, function args
- Fast, automatic
 - Done by the compiler
 - Abstracts away any concept of allocating/de-allocating

Heap implementations

- Tons of different heap implementations
 - dlmalloc
 - ptmalloc
 - tcmalloc
 - jemalloc
 - nedmalloc
 - Hoard
- Some applications even create their own heap implementations!

Heap implementations

- glibc 2.19 is what we have on the Warzone –
Default for Ubuntu 14.04 (32bit)
- Its heap implementation is based on ptmalloc2 –
Very fast, low fragmentation, thread safe

Heap Chunks

```
unsigned int * buffer = NULL;  
buffer = malloc(0x100);
```

// Out comes a heap chunk

Heap Chunk

Previous Chunk Size
(4 bytes)

Chunk Size
(4 bytes)

Flags

Data
(8 + (n / 8) * 8 bytes)

$\ast (buffer - 2)$

$\ast (buffer - 1)$

$\ast buffer$

```
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], ebx  
jnz     short loc_313066  
mov     eax, [ebp+var_70]  
cmp     eax, [ebp+var_84]  
jb      short loc_313066  
sub     eax, [ebp+var_84]  
push    esi  
push    esi  
push    eax  
push    edi  
mov     [ebp+arg_0], eax  
call    sub_31486A  
test    eax, eax  
jz      short loc_31306D  
push    esi  
lea     eax, [ebp+arg_0]  
push    eax  
mov     esi, 1D0h  
push    esi  
push    [ebp+arg_4]  
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], esi  
jz      short loc_31306F
```

```
call    sub_3140F3  
jmp     short loc_31308C
```

loc_31307D:

CODE XREF: sub_312FD8

```
mov     [ebp+var_4], eax
```

Malloc Trivia

- **malloc(28);**
 - 40 bytes
- **malloc(4);**
 - 16 bytes
- **malloc(10);**
 - 24 bytes
- **malloc(0);**
 - 16 bytes

How many bytes on the heap are your malloc chunks really taking up?

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
push    eax
push    edi
mov     [ebp+arg_0], eax
call    sub_314623
test    eax, eax
jz      short loc_31306D
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 1D0h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31306F

loc_313066:                                     ; CODE XREF: sub_312FD8
                                              ; sub_312FD8+56
push    0Dh
call    sub_31411B

loc_31306D:                                     ; CODE XREF: sub_312FD8
                                              ; sub_312FD8+49
call    sub_3140F3
test    eax, eax
jg      short loc_31307D
call    sub_3140F3
jmp     short loc_31308C

; -----
loc_31307D:                                     ; CODE XREF: sub_312FD8
                                              ; sub_312FD8+56
mov     [ebp+var_4], eax
mov     [ebp+var_4], eax
```

Heap Chunks

- Flags

- Because of byte alignment, the lower 3 bits of the chunk size field would always be zero. Instead they are used for flag bits.

0x01 **PREV_INUSE** – set when previous chunk is in use

0x02 **IS_MAPPED** – set if chunk was obtained with `mmap()`

0x04 **NON_MAIN_ARENA** – set if chunk belongs to a thread arena

Heap Chunk

Previous Chunk Size
(4 bytes)

Chunk Size
(4 bytes)

Flags

Data
(8 + (n / 8) * 8 bytes)

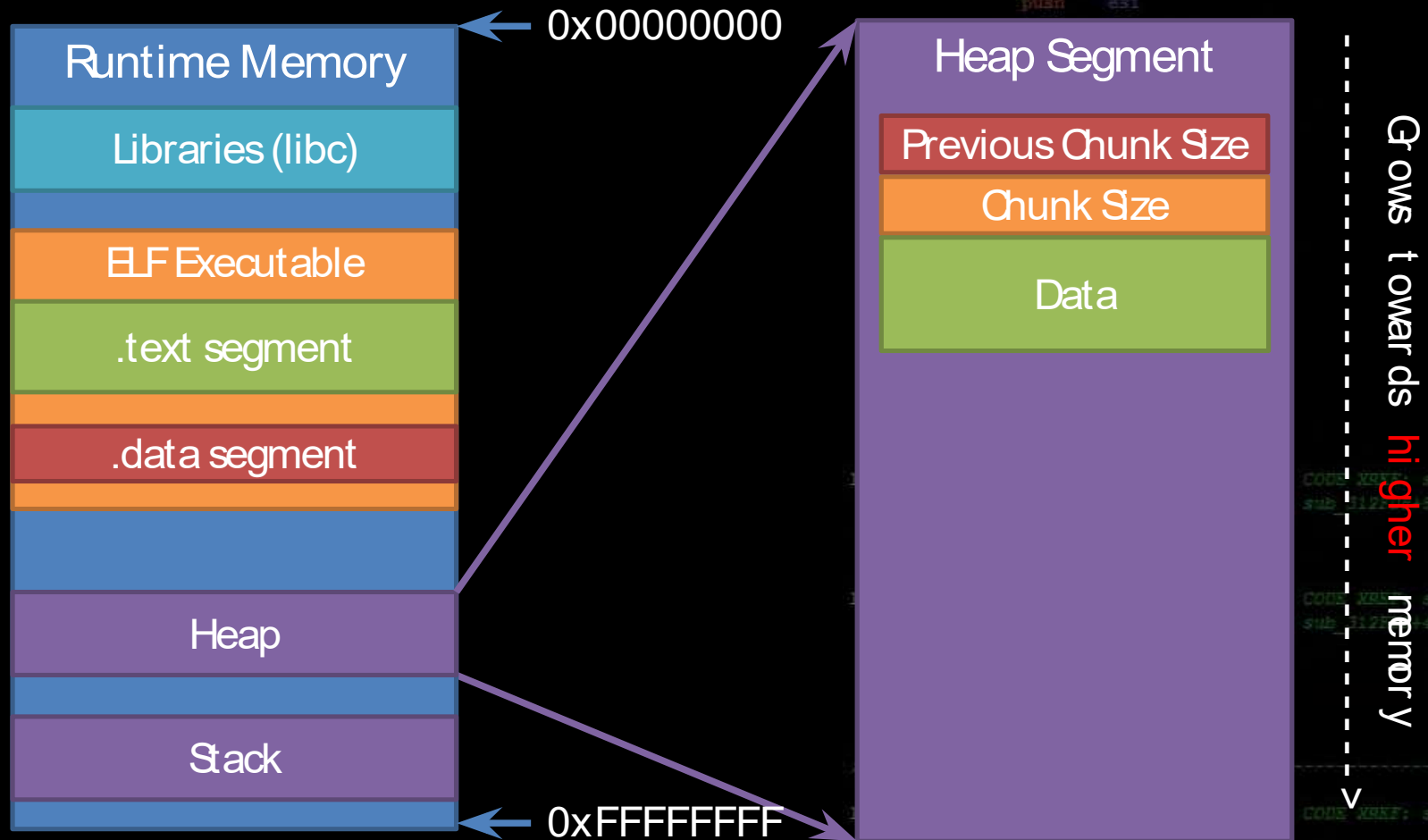
$*(\text{buffer} - 2)$

$*(\text{buffer} - 1)$

$*\text{buffer}$

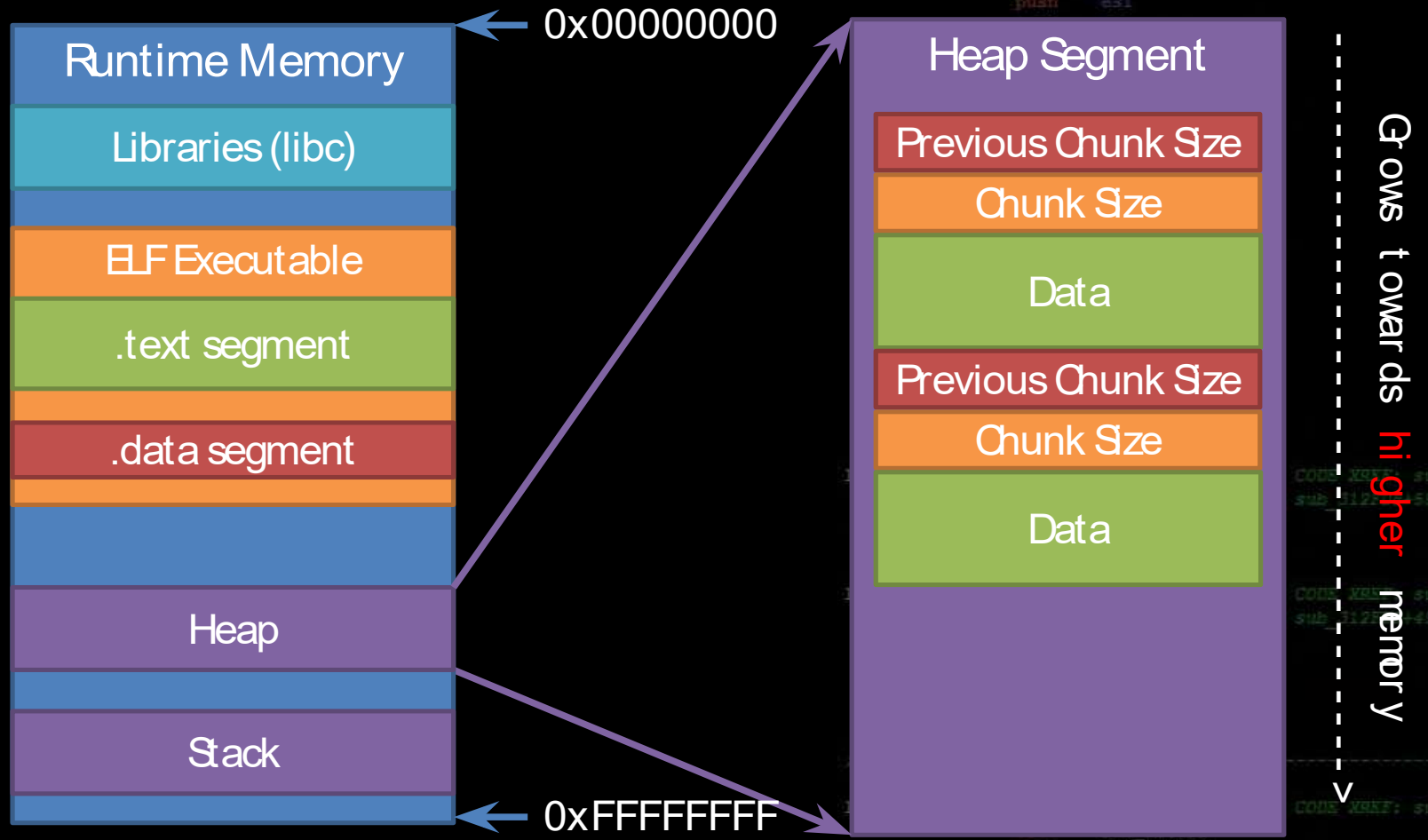
Heap Allocations

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
```



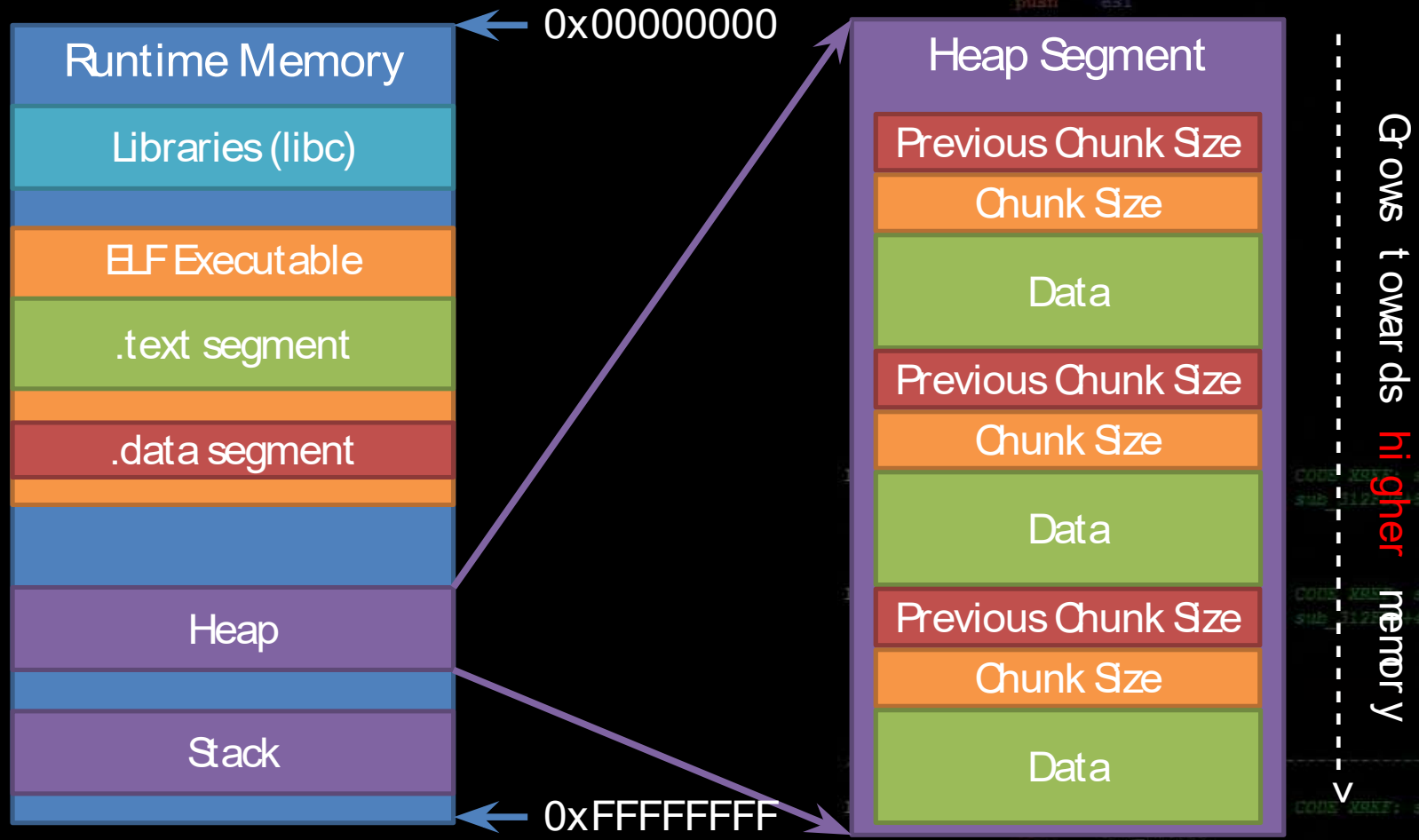
Heap Allocations

```
push    edi
call    sub_314623
test    eax, eax
jz       short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
```



Heap Allocations

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
```



Heap Chunks – In Use

- Heap chunks exist in two states
 - in use (malloc'd)
 - free'd

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
push    eax
push    edi
mov     [ebp+arg_0], eax
call    sub_31486A
test    eax, eax
jz      short loc_31306D
push    esi
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 1D0h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31306F
```

Heap Chunk

Previous Chunk Size
(4 bytes)

Chunk Size
(4 bytes)

Flags

Data
(8 + (n / 8) * 8 bytes)

* (buf f er - 2)

* (buf f er - 1)

* buf f er

```
call    sub_3140F3
jmp     short loc_31308C
```

loc_31307D:

```
call    sub_3140F3
```

```
mov     [ebp+var_4], eax
```

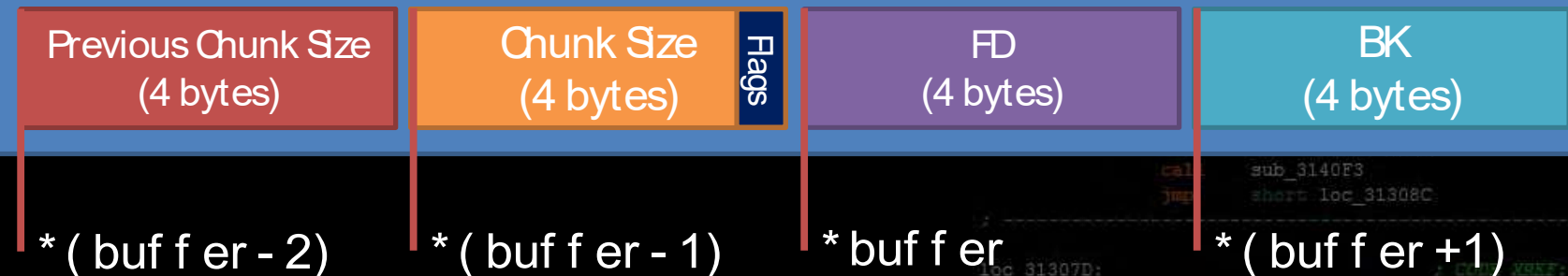
Heap Chunks – Freed

`free( buffer );`

- **Forward Pointer**
 - A pointer to the next freed chunk
- **Backwards Pointer**
 - A pointer to the previous freed chunk

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
push    eax
push    edi
mov     [ebp+arg_0], eax
call    sub_31486A
test    eax, eax
jz      short loc_31306D
push    esi
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 1D0h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31306F
```

Heap Chunk (freed)



```
call    sub_3140F3
jmp     short loc_31308C
loc_31307D:
call    sub_3140F3
mov     [ebp+var_4], eax
```

Heap exploitation

- Heap Overflows
- Use After Free
- Heap Spraying

Heap Overflows

- Buffer overflows are basically the same on the **heap** as they are on the **stack**
- **Heap** cookies/ canaries aren't a thing
 - No 'return' addresses to protect

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
push    eax
push    esi
call    sub_314623
call    sub_31486A
test    eax, eax
jz      short loc_31306D
push    esi
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 1D0h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31306F

loc_313066:                                     ; CODE XREF: sub_312FD8
                                              ; sub_312FD8+56
push    0Dh
call    sub_31411B

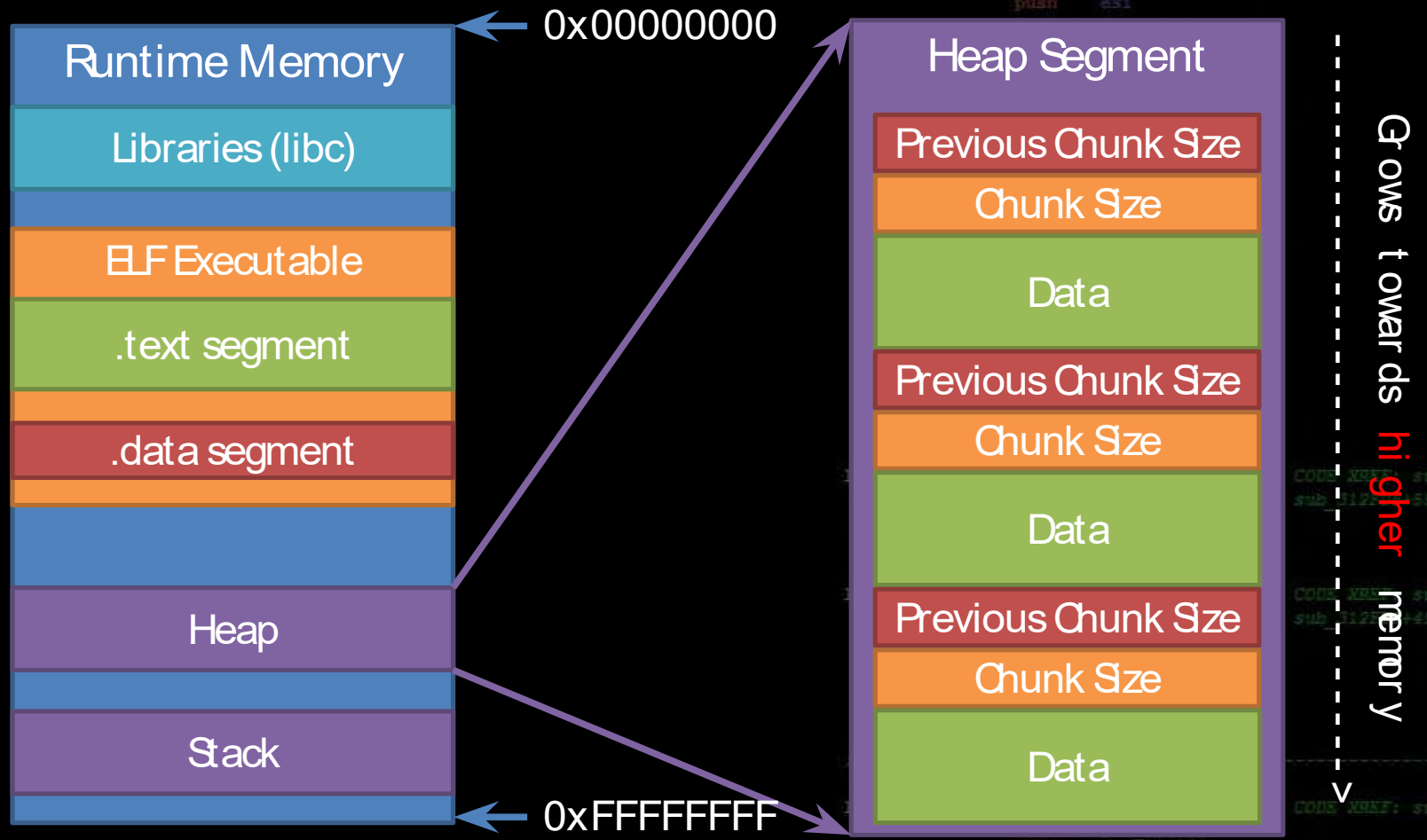
loc_31306D:                                     ; CODE XREF: sub_312FD8
                                              ; sub_312FD8+49
call    sub_3140F3
test    eax, eax
jg      short loc_31307D
call    sub_3140F3
jmp     short loc_31308C

; -----
loc_31307D:                                     ; CODE XREF: sub_312FD8
call    sub_3140F3

mov     [ebp+var_41], eax
```

Heap Overflows

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
```



```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
```



Heap Overflows

- In the real world, lots of cool and complex things like objects/structs end up on the heap
 - Anything that handles the data you just corrupted is now viable attack surface in the application
- It's common to put function pointers in structs which generally are malloc'd on the heap
 - Overwrite a function pointer on the heap, and force a codepath to call that object's function!

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
push    eax
push    esi
call    sub_31486A
test    eax, eax
jz      short loc_31306D
lea     eax, [ebp+arg_0]
push    eax
push    esi
push    [ebp+arg_4]
push    esi
call    sub_314012
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31306F
```

```
loc_31306F:                                ; CODE XREF: sub_312FD8
push    esi
call    sub_31411B
loc_313070:                                ; CODE XREF: sub_312FD8
call    sub_3140F3
test    eax, eax
jz      short loc_313070
call    sub_3140F3
jmp     short loc_31308C
loc_31307D:                                ; CODE XREF: sub_312FD8
call    sub_3140F3
```

Heap Overflows

```
struct t_oyst_r {  
    void (* message)(char*);  
    char buffer[20];  
};
```

```
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], ebx  
jnz     short loc_313066  
mov     eax, [ebp+var_70]  
cmp     eax, [ebp+var_84]  
jb      short loc_313066  
sub     eax, [ebp+var_84]  
push    esi  
push    esi  
push    eax  
push    edi  
mov     [ebp+arg_0], eax  
call    sub_31486A  
test    eax, eax  
jz      short loc_31306D  
push    esi  
lea     eax, [ebp+arg_0]  
push    eax  
mov     esi, 1D0h  
push    esi  
push    [ebp+arg_4]  
push    edi  
call    sub_314623  
mov     eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], esi  
jz      short loc_31306F
```

```
loc_313066:                                     ; CODE XREF: sub_312FD8  
                                                ; sub_312FD8+56
```

```
push    0Dh  
call    sub_31411B
```

```
loc_31306D:                                     ; CODE XREF: sub_312FD8  
                                                ; sub_312FD8+49
```

```
call    sub_3140F3  
test    eax, eax  
jg      short loc_31307D  
call    sub_3140F3  
jmp     short loc_31308C
```

```
loc_31307D:                                     ; CODE XREF: sub_312FD8
```

```
call    sub_3140F3
```

```
mov     [ebp+var_41], eax
```

Heap Overflows

```
cool guy = malloc(sizeof(struct t_oyster));  
lame guy = malloc(sizeof(struct t_oyster));
```

```
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], ebx  
jnz     short loc_313066  
mov     eax, [ebp+var_70]  
cmp     eax, [ebp+var_84]  
jb      short loc_313066  
sub     eax, [ebp+var_84]  
push    esi  
push    esi  
push    eax  
push    edi  
mov     [ebp+arg_0], eax  
call    sub_31486A  
test    eax, eax  
jz      short loc_31306D  
push    esi
```

Heap Overflows

```
cool_guy = malloc(sizeof(struct t_oyster));  
lame_guy = malloc(sizeof(struct t_oyster));
```

```
cool_guy->message = &print_cool;  
lame_guy->message = &print_meh;
```

```
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], ebx  
jnz     short loc_313066  
mov     eax, [ebp+var_70]  
cmp     eax, [ebp+var_84]  
jb      short loc_313066  
sub     eax, [ebp+var_84]  
push    esi  
push    esi  
push    eax  
push    edi  
mov     [ebp+arg_0], eax  
call    sub_31486A  
test    eax, eax  
jz      short loc_31306D  
push    esi  
lea     eax, [ebp+arg_0]  
push    eax
```

```
mov     [ebp+var_4], eax
```

Heap Overflows

The C library function **strcspn()** calculates the length of the number of characters before the 1st occurrence of character present in the string.

```
cool_guy = malloc(sizeof(struct t_oyst));
lameguy = malloc(sizeof(struct t_oyst));
```

```
cool_guy->message = &print_cool;
lameguy->message = &print_meh;
```

```
printf("Input cool guy's name: ");
fgets(cool_guy->buffer, 200, stdin); // oopz...
cool_guy->buffer[strcspn(cool_guy->buffer, "\n")] = 0;
```

Silly heap overflow

```
push    esi
push    esi
push    eax
push    edi
mov     [ebp+arg_0], eax
call    sub_31486A
test    eax, eax
jz      short loc_31306D
push    esi
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 1D0h
push    esi
mov     [ebp+arg_1], esi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31306F
loc_313066:
; CODE XREF: sub_312FD8
mov     [ebp+var_4], eax
```

Heap Overflows

The C library function **strcspn()** calculates the length of the number of characters before the 1st occurrence of character present in the string.

```
cool_guy = malloc(sizeof(struct t_oyst));
I_am_guy = malloc(sizeof(struct t_oyst));
```

```
cool_guy->message = &print_cool;
I_am_guy->message = &print_meh;
```

```
printf("Input cool_guy's name: ");
fgets(cool_guy->buffer, 200, stdin); // oopz...
cool_guy->buffer[strcspn(cool_guy->buffer, "\n")] = 0;
```

```
printf("Input I_am_guy's name: ");
fgets(I_am_guy->buffer, 20, stdin);
I_am_guy->buffer[strcspn(I_am_guy->buffer, "\n")] = 0;
```

Silly heap overflow

```
push    esi
push    esi
push    eax
push    edi
mov     [ebp+arg_0], eax
call    sub_31486A
test    eax, eax
jz      loc_31306D
push    esi
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 1D0h
push    esi
mov     [ebp+arg_1], esi
call    sub_314623
test    eax, eax
short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31306F
loc_313066:
; CODE XREF: sub_312FD8
; sub_312FD8+56
push    0Dh
call    sub_31411B
loc_31306D:
; CODE XREF: sub_312FD8
; sub_312FD8+49
call    sub_3140F3
test    eax, eax
mov     [ebp+var_4], eax
```

Heap Overflows

The C library function `strcspn()` calculates the length of the number of characters before the 1st occurrence of character present in the string.

```
cool_guy = malloc(sizeof(struct t_oyst));
I_am_cool_guy = malloc(sizeof(struct t_oyst));
```

```
cool_guy->message = &print_cool;
I_am_cool_guy->message = &print_meh;
```

```
printf("Input cool_guy's name: ");
fgets(cool_guy->buffer, 200, stdin); // oopz...
cool_guy->buffer[strcspn(cool_guy->buffer, "\n")] = 0;
```

```
printf("Input I_am_cool_guy's name: ");
fgets(I_am_cool_guy->buffer, 20, stdin);
I_am_cool_guy->buffer[strcspn(I_am_cool_guy->buffer, "\n")] = 0;
```

```
cool_guy->message(cool_guy->buffer);
I_am_cool_guy->message(I_am_cool_guy->buffer);
```

Silly heap overflow

Overwritten
function pointer!

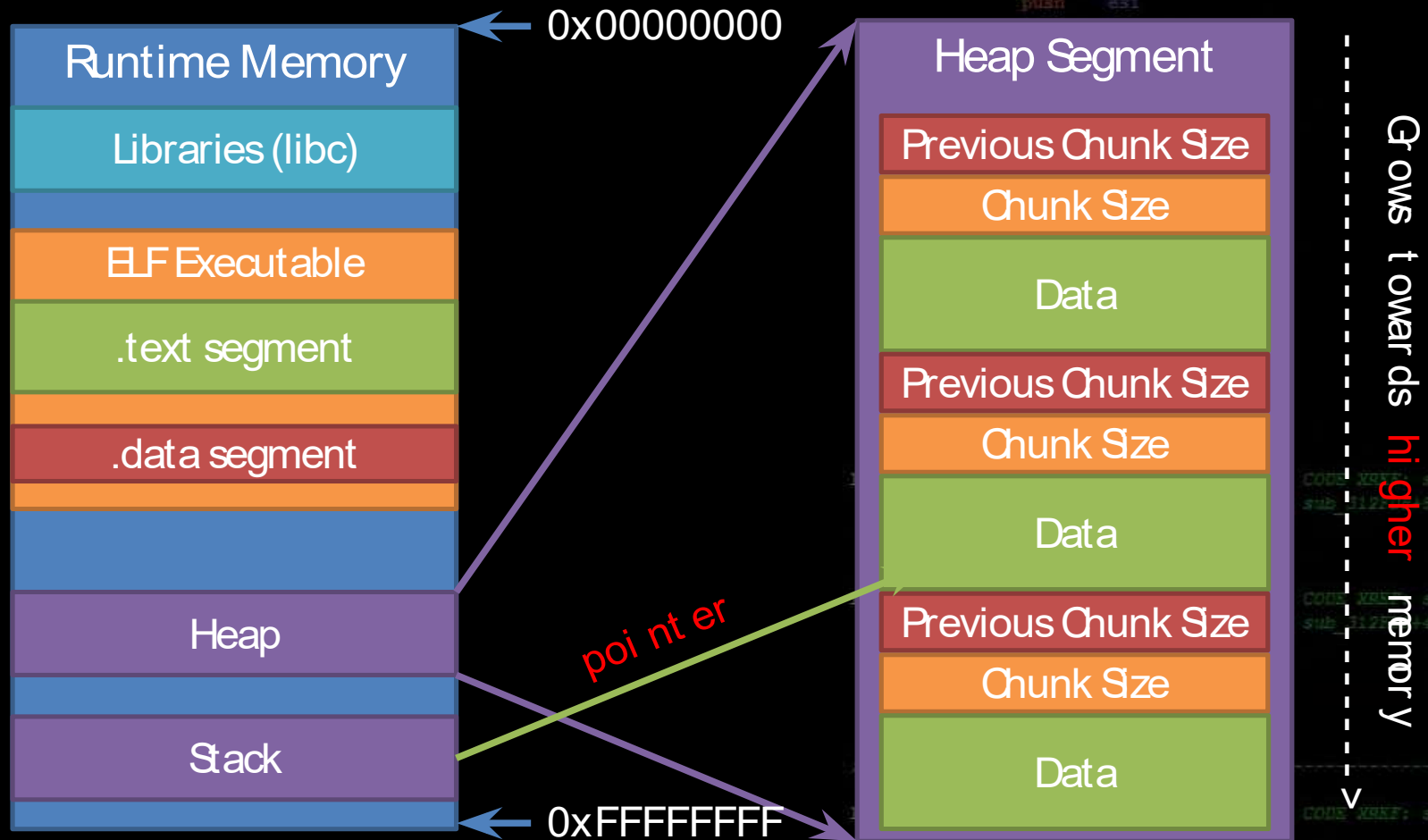
```
push    esi
push    esi
push    eax
push    edi
mov     [ebp+arg_0], eax
call    sub_31486A
test    eax, eax
jz      short loc_31306D
push    esi
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 1D0h
push    esi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31306F
loc_313066:
; CODE XREF: sub_312FD8
; sub_312FD8+56
push    0Dh
call    sub_31411B
loc_31306D:
; CODE XREF: sub_312FD8
; sub_312FD8+49
call    sub_3140F3
test    eax, eax
jz      short loc_31307D
call    sub_3140F3
loc_31307D:
; CODE XREF: sub_312FD8
; sub_312FD8+49
call    sub_3140F3
mov     [ebp+var_4], eax
```

Use-after-free attacks

- A class of vulnerability where **data on the heap is freed**, but **a leftover reference or 'dangling pointer' is used by the code** as if the data were still valid
- Most popular in Web Browsers, complex programs
- Also known as UAF

Use After Free

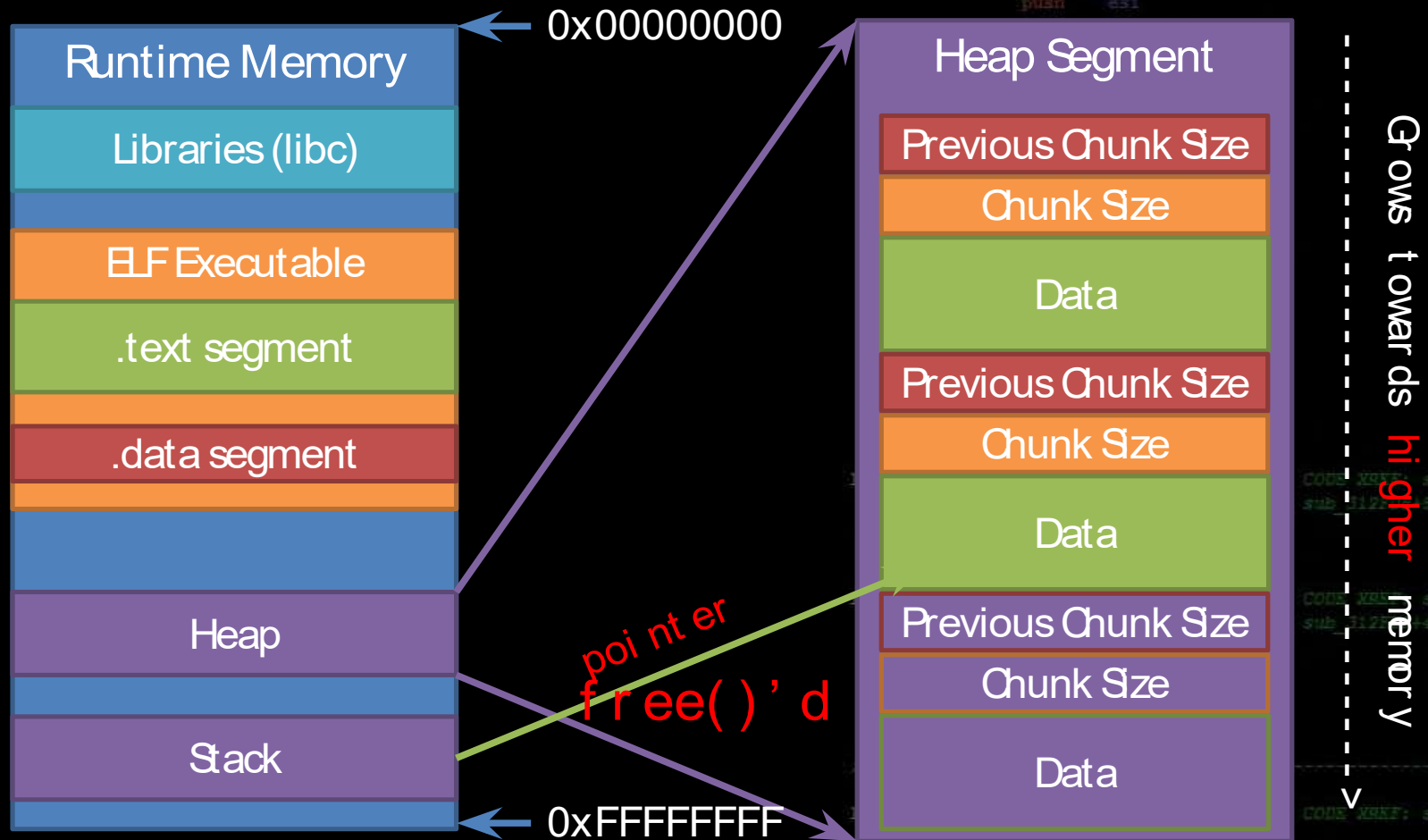
```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
```



Use After Free

```

push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
    
```

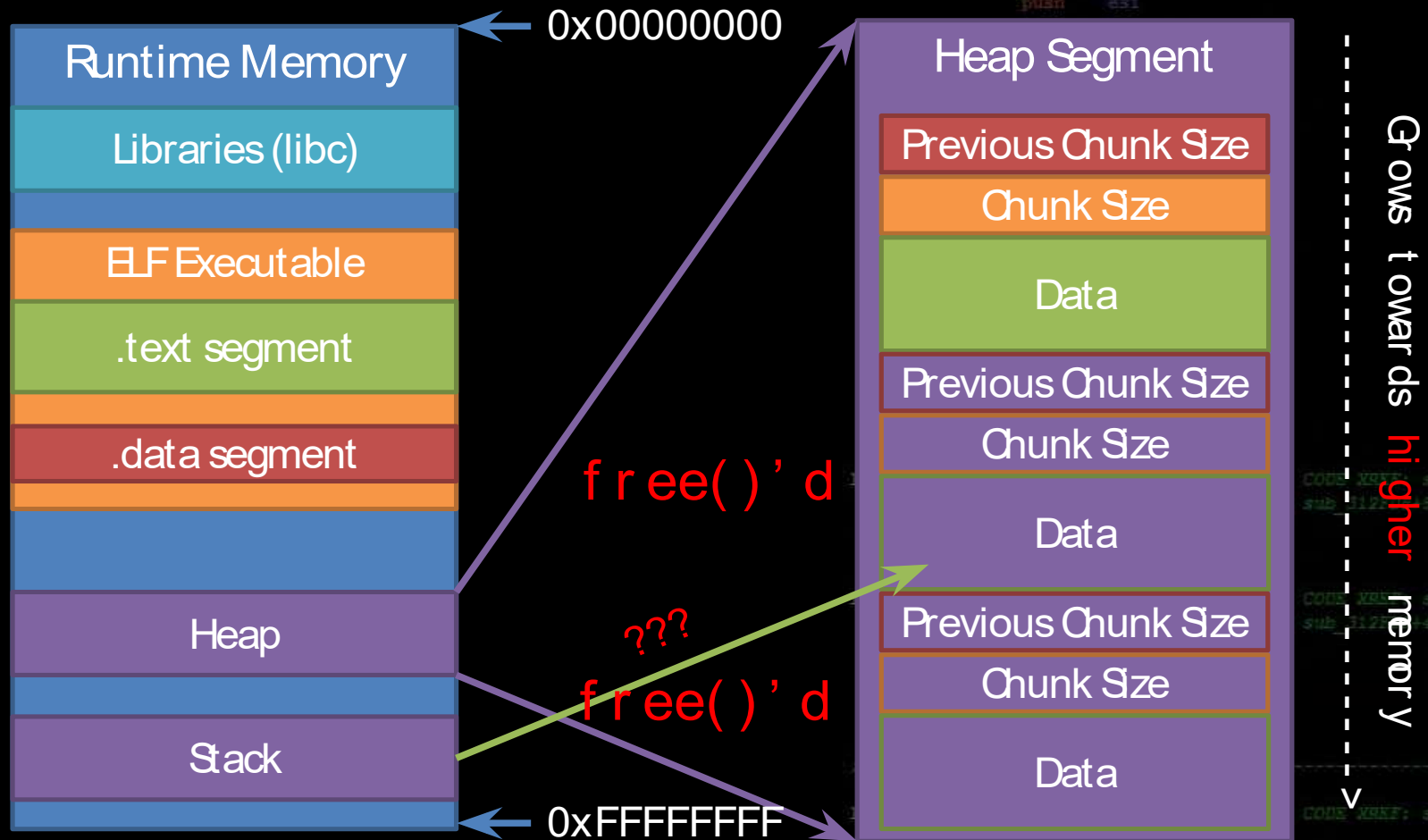


Use After Free

```

push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi

```



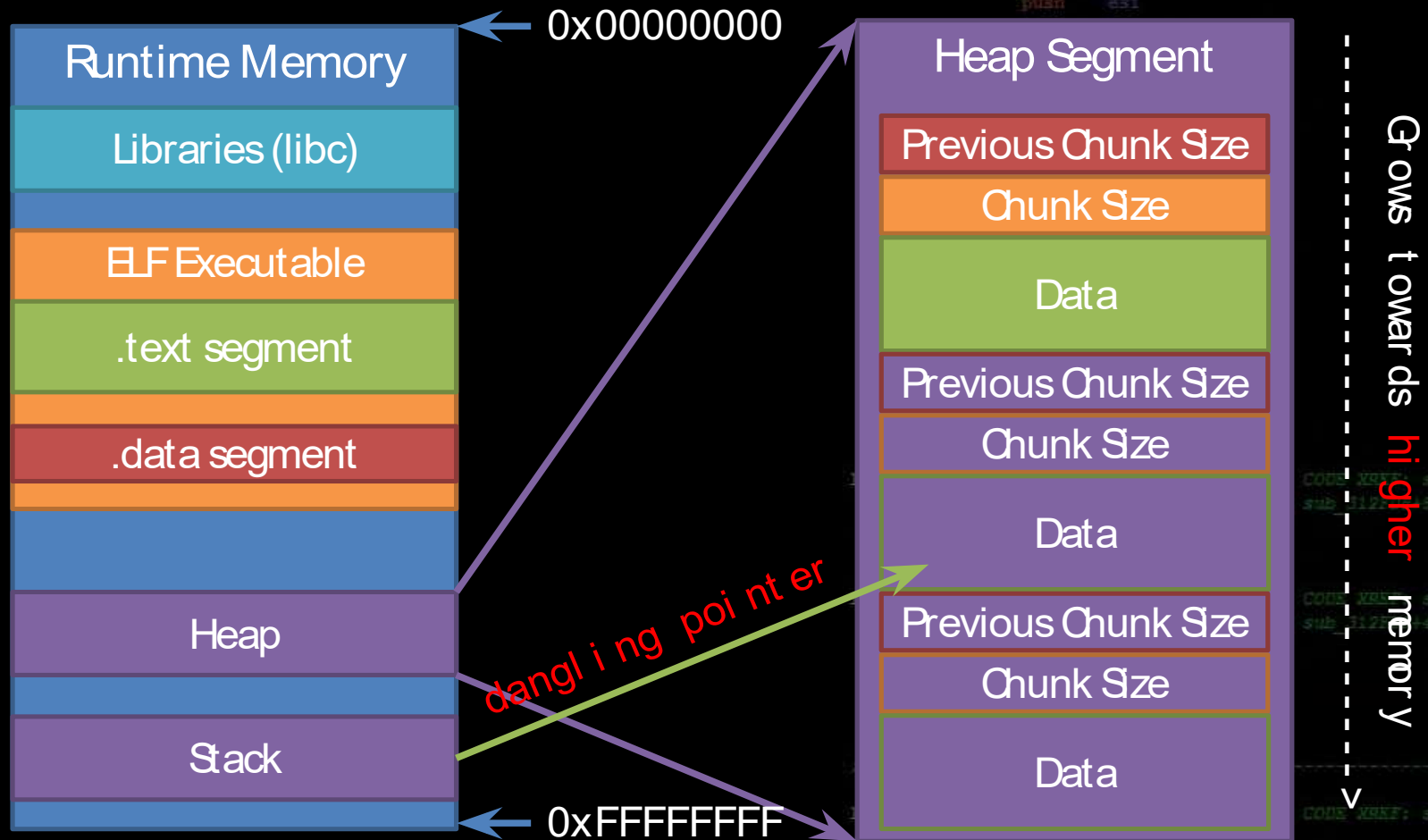
```

CODE_XREF: sub_312FD8
sub_312FD8: 55
CODE_XREF: sub_312FD8
sub_312FD8: 49
CODE_XREF: sub_312FD8

```

Use After Free

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi
```



Dangling pointer

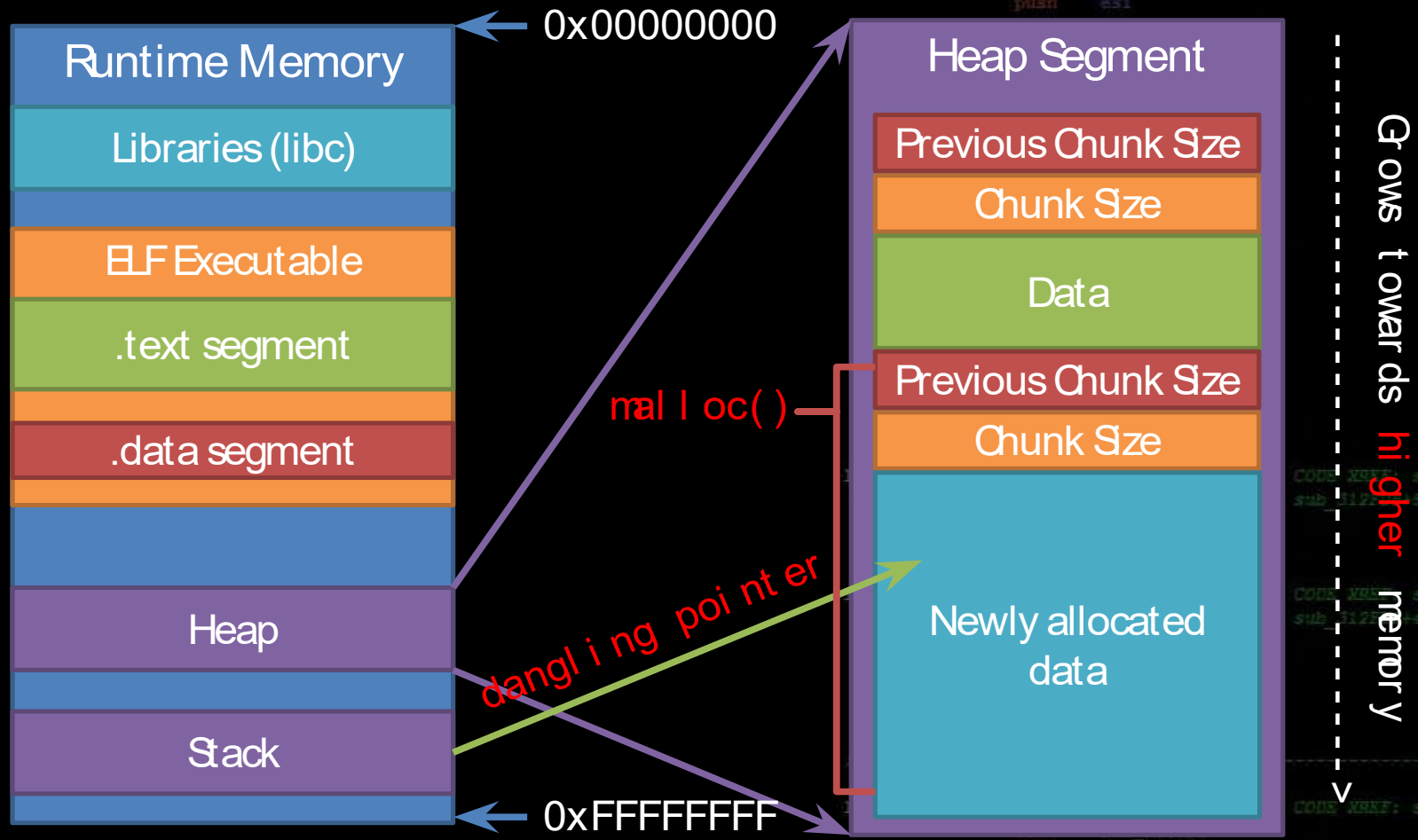
- A leftover pointer in your code that references free'd data and is prone to be re-used
- As the memory it's pointing at was freed, there's no guarantees on what data is there now
- Also known as stale pointer, wild pointer

Use After Free

```

push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
push    esi

```



```

CODE_XREF: sub_312FD8
sub_312FD8: 55
CODE_XREF: sub_312FD8
sub_312FD8: 49
CODE_XREF: sub_312FD8

```

Exploiting a Use After Free

- To exploit a **UAF**, you usually have to allocate a different type of object over the one you just freed

```
struct toystor {  
    void (* message)(char *);  
    char buffer[20];  
};
```

```
struct person {  
    int favorite_num;  
    int age;  
    char name[16];  
};
```

```
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], ebx  
jnz     short loc_313066  
mov     eax, [ebp+var_70]  
mov     eax, [ebp+var_84]  
jb      short loc_313066  
sub     eax, [ebp+var_84]  
push    esi
```

```
push    esi  
push    eax  
push    edi  
call    sub_31486A  
test    eax, eax  
jz      short loc_31306D  
lea     eax, [ebp+arg_0]  
push    eax  
mov     esi, 1D0h  
push    esi  
push    [ebp+arg_4]  
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
mov     [ebp+arg_0], esi  
jz      short loc_31306F
```

```
loc_313066: ; CODE XREF: sub_312FD8  
; sub_312FD8+56
```

```
loc_31411B: ; CODE XREF: sub_31411B
```

```
loc_312FD8: ; CODE XREF: sub_312FD8  
; sub_312FD8+49
```

```
call    sub_3140F3  
test    eax, eax  
jg      short loc_31307D  
call    sub_3140F3  
jmp     short loc_31308C
```

```
loc_31307D: ; CODE XREF: sub_312FD8  
call    sub_3140F3
```

```
mov     [ebp+var_41], eax
```

Exploiting a Use After Free

- To exploit a **UAF**, you usually have to allocate a different type of object over the one you just freed **assume dangling pointer exists**

1. free()

```
struct toystor {  
    void (* message)(char *);  
    char buffer[20];  
};
```

```
struct person {  
    int favorite_num;  
    int age;  
    char name[16];  
};
```

```
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], ebx  
short   loc_313066  
eax, [ebp+var_70]  
eax, [ebp+var_84]  
jb      short loc_313066  
sub     eax, [ebp+var_84]  
push    esi
```

```
push    esi  
push    eax  
push    edi  
call    sub_31486A  
test    eax, eax  
jz      short loc_31306D  
lea     eax, [ebp+arg_0]  
push    eax  
mov     esi, 1D0h  
push    esi
```

```
push    [ebp+arg_4]  
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
jz      [ebp+arg_0], esi  
short   loc_31306F
```

```
loc_313066: CODE XREF: sub_312FD8  
; sub_312FD8+56
```

```
00h  
sub_31411B
```

```
loc_313067: CODE XREF: sub_312FD8  
; sub_312FD8+49
```

```
call    sub_3140F3  
test    eax, eax  
jg      short loc_31307D  
call    sub_3140F3  
jmp     short loc_31308C
```

```
loc_31307D: CODE XREF: sub_312FD8  
call    sub_3140F3
```

```
mov     [ebp+var_41], eax
```

Exploiting a Use After Free

- To exploit a **UAF**, you usually have to allocate a different type of object over the one you just freed

1. **free()**

```
struct toystor {  
    void (* message)(char *);  
    char buffer[20];  
};
```

2. **malloc()**

```
struct person {  
    int favorite_num;  
    int age;  
    char name[16];  
};
```

```
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], ebx  
short   loc_313066  
eax, [ebp+var_70]  
eax, [ebp+var_84]  
jb      short loc_313066  
sub     eax, [ebp+var_84]  
push    esi
```

```
push    esi  
push    eax  
push    esi  
call    sub_31486A  
test    eax, eax  
jz      short loc_31306D  
lea     eax, [ebp+arg_0]  
push    eax  
mov     esi, 1D0h  
push    esi  
push    [ebp+arg_4]  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
jz      short loc_31306F
```

```
loc_313066: CODE XREF: sub_312FD8  
; sub_312FD8+56
```

```
0Dh  
sub_31411B
```

```
loc_31306F: CODE XREF: sub_312FD8  
; sub_312FD8+49
```

```
call    sub_3140F3  
test    eax, eax  
jg      short loc_31307D  
call    sub_3140F3  
jmp     short loc_31308C
```

```
loc_31307D: CODE XREF: sub_312FD8  
call    sub_3140F3
```

```
mov     [ebp+var_41], eax
```

Exploiting a Use After Free

- To exploit a **UAF**, you usually have to allocate a different type of object over the one you just freed

1. **free()**

```
struct t_oys_tr {  
    void (* message)(char *);  
    char buffer[20];  
};
```

2. **malloc()**

```
struct person {  
    int favorite_num;  
    int age;  
    char name[16];  
};
```

3. Set **favorite_num** = 0x41414141

```
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], ebx  
short  loc_313066  
eax, [ebp+var_70]  
eax, [ebp+var_84]  
jb      short loc_313066  
sub     eax, [ebp+var_84]  
push    esi
```

```
push    esi  
push    eax  
push    esi  
call    sub_31486A  
test    eax, eax  
jz      short loc_31306D  
lea     eax, [ebp+arg_0]  
push    eax  
mov     esi, 1D0h  
push    esi  
push    [ebp+arg_4]  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
[ebp+arg_0], esi  
jz      short loc_31306F
```

```
loc_313066: CODE XREF: sub_312FD8  
; sub_312FD8+56
```

```
int age; 0Dh  
sub_31411B
```

```
char name[16]; CODE XREF: sub_312FD8  
; sub_312FD8+49
```

```
call    sub_3140F3  
test    eax, eax  
jg      short loc_31307D  
call    sub_3140F3  
jz      short loc_3140C0
```

```
loc_31307D: CODE XREF: sub_312FD8  
call    sub_3140F3
```

```
mov     [ebp+var_41], eax
```

Exploiting a Use After Free

- To exploit a **UAF**, you usually have to allocate a different type of object over the one you just freed

1. **free()**

```
struct toystor {  
    void (* message)(char *);  
    char buffer[20];  
};
```

4. Force dangling pointer to call 'message()'

2. **malloc()**

```
struct person {  
    int favorite_num;  
    int age;  
    char name[16];  
};
```

3. Set **favorite_num** = 0x41414141

```
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], ebx  
short loc_313066  
eax, [ebp+var_70]  
eax, [ebp+var_84]  
jb      short loc_313066  
sub     eax, [ebp+var_84]  
push    esi
```

```
push    esi  
push    eax  
push    esi  
call    sub_31486A  
test    eax, eax  
jz      short loc_31306D  
lea     eax, [ebp+arg_0]  
push    eax  
mov     esi, 1D0h  
push    esi  
push    [ebp+arg_4]  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
[ebp+arg_0], esi  
jz      short loc_31306F
```

```
loc_313066: int favorite_num; CODE XREF: sub_312FD8  
; sub_312FD8+56
```

```
int age; 0Dh  
sub_31411B
```

```
loc_31306F: char name[16]; CODE XREF: sub_312FD8  
; sub_312FD8+49
```

```
call    sub_3140F3  
test    eax, eax  
jg      short loc_31307D  
call    sub_3140F3  
jz      short loc_31307D
```

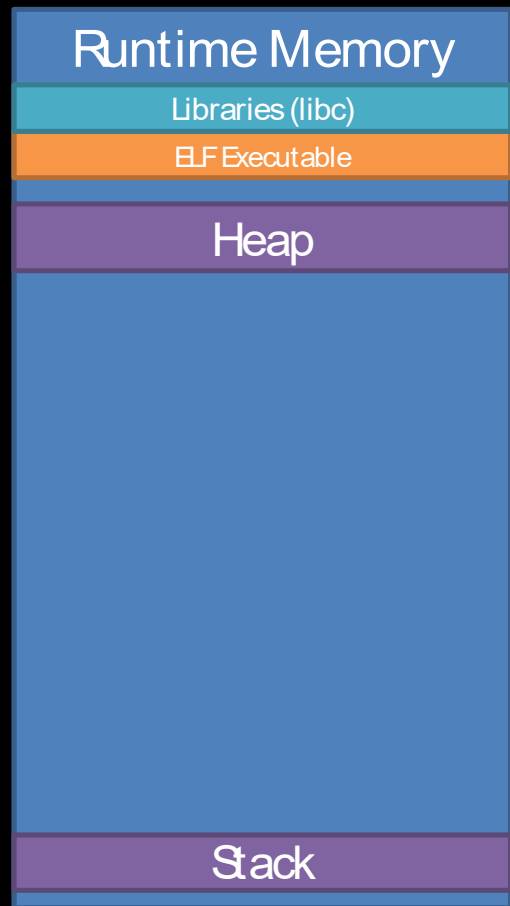
```
loc_31307D: CODE XREF: sub_312FD8  
call    sub_3140F3
```

```
mov     [ebp+var_41], eax
```

Heap Spraying

- A technique used to **increase exploit reliability**, by **filling the heap with large chunks of data relevant to the exploit** you're trying to land
- It can assist with bypassing ASLR
- A heap spray is **not a vulnerability or security flaw**

Heap Spray in Action



← 0x00000000 – Start of memory

← 0x08048000 – .text Segment in ELF

← 0x09104000 – Top of heap

```

filler = "AAAAAAAAAAAAA.-.-";
for(i = 0; i < 3000; i++)
{
    temp = malloc(1000000);
    memcpy(temp, filler, 1000000);
}
    
```

← 0xbffff000 – Top of stack

← 0xFFFFFFFF – End of memory

```

push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
    
```

```

push    esi
push    edi
mov     [ebp+arg_0], eax
call    sub_31486A
test    eax, eax
jz      short loc_31306D
lea     eax, [ebp+arg_0]
push    esi, 1D0h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
    
```

```

mov     eax, [ebp+arg_0]
jz      short loc_31306E
[ebp+arg_0], esi
short loc_31306F
    
```

```

loc_313066:                                ; CODE XREF: sub_312FD8
; sub_312FD8+56
    
```

```

push    esi
call    sub_314118
call    sub_314118
loc_31306E:                                ; CODE XREF: sub_312FD8+58
; sub_312FD8+49
    
```

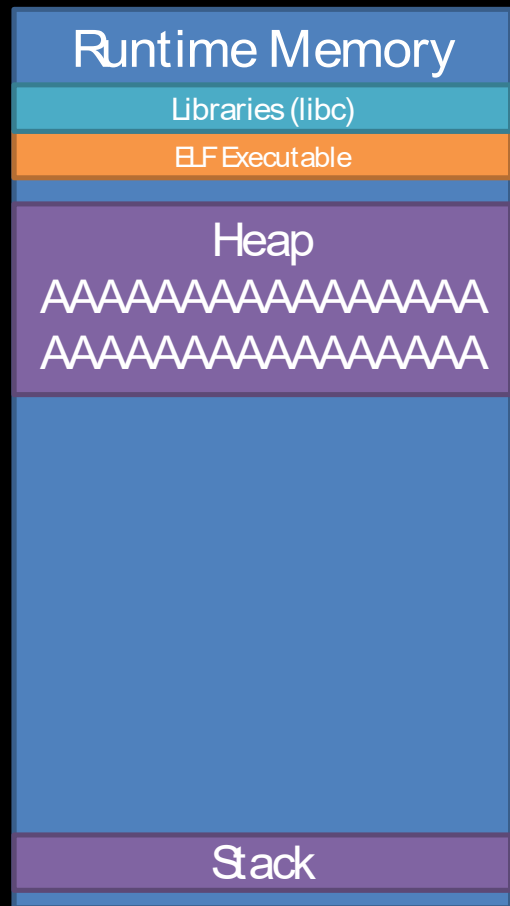
```

call    sub_3140F3
test    eax, eax
jg      short loc_31307D
call    sub_3140F3
jmp     short loc_31308C
    
```

```

loc_31307D:                                ; CODE XREF: sub_312FD8
; sub_312FD8+5A
    
```

Heap Spray in Action



← 0x00000000 – Start of memory

← 0x08048000 – .text Segment in ELF

← 0x09104000 – Top of heap

```
filler = "AAAAAAAAAAAAAAAA";
for(i = 0; i < 3000; i++)
{
    temp = malloc(1000000);
    memcpy(temp, filler, 1000000);
}
```

← 0xbffff000 – Top of stack

← 0xFFFFFFFF – End of memory

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
```

```
push    esi
push    edi
mov     [ebp+arg_0], eax
call    sub_31486A
test    eax, eax
```

```
loc_31306D:
mov     eax, [ebp+arg_0]
push    esi, 1D0h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
```

```
mov     eax, [ebp+arg_0]
jz      short loc_31306E
[ebp+arg_0], esi
short loc_31306F
```

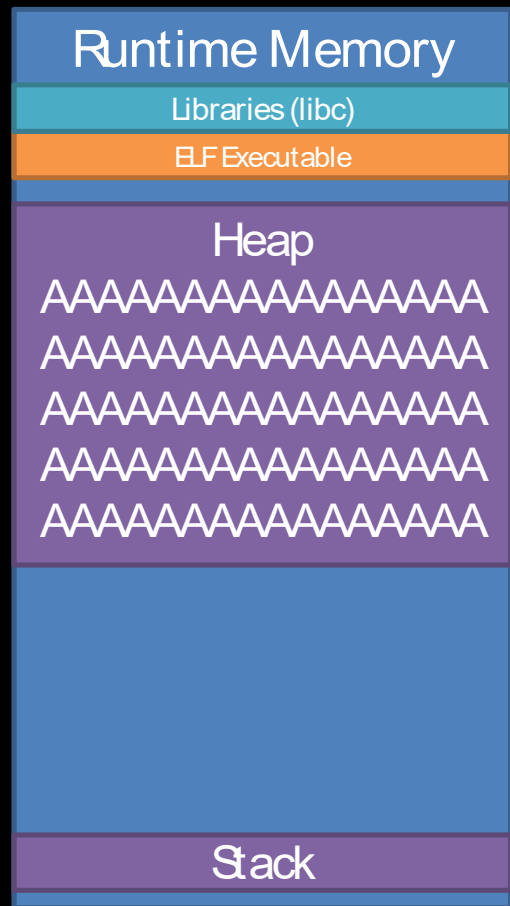
```
loc_313066: ; CODE XREF: sub_312FD8
; sub_312FD8+56
```

```
loc_31306E:
call    sub_314118
loc_31306F: ; CODE XREF: sub_312FD8+49
```

```
call    sub_3140F3
test    eax, eax
jg      short loc_31307D
call    sub_3140F3
jmp     short loc_31308C
```

```
loc_31307D: ; CODE XREF: sub_312FD8
; sub_312FD8+56
```

Heap Spray in Action



← 0x00000000 – Start of memory

← 0x08048000 – .text Segment in ELF

← 0x09104000 – Top of heap

```

filler = "AAAAAAAAAAAAAAAAAAAA";
for(i = 0; i < 3000; i++)
{
    temp = malloc(1000000);
    memcpy(temp, filler, 1000000);
}
    
```

← 0xbffff000 – Top of stack

← 0xFFFFFFFF – End of memory

```

push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
    
```

```

push    esi
push    edi
mov     [ebp+arg_0], eax
call    sub_31486A
test    eax, eax
jz      short loc_31306D
lea     eax, [ebp+arg_0]
mov     esi, 1D0h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
    
```

```

filler = "AAAAAAAAAAAAAAAAAAAA";
for(i = 0; i < 3000; i++)
{
    loc_313066:
    temp = malloc(1000000);
    memcpy(temp, filler, 1000000);
}
    
```

```

test    eax, eax
jz      short loc_31306E
mov     [ebp+arg_0], esi
jnz     short loc_31306F

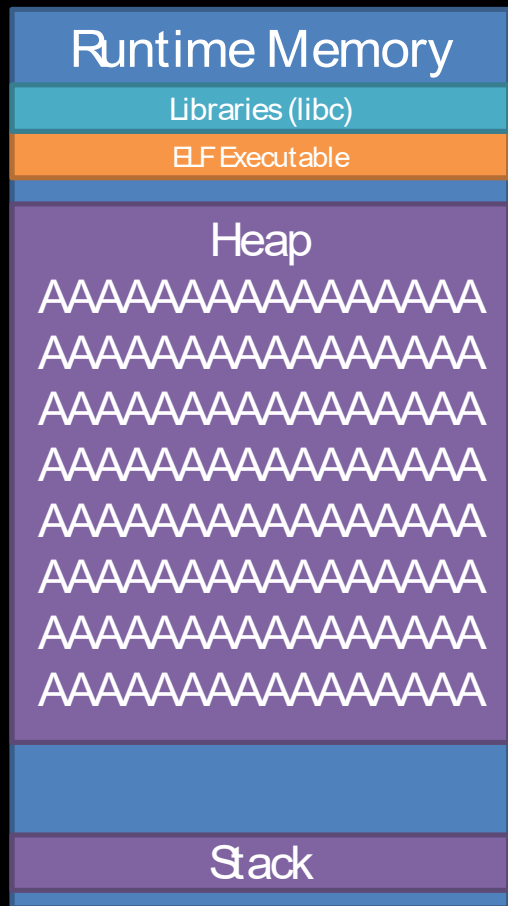
loc_313066:
; CODE XREF: sub_312FD8+56
; sub_312FD8+56
push    esi
call    sub_314118
; CODE XREF: sub_312FD8+56
; sub_312FD8+56
call    sub_3140F3
test    eax, eax
jg      short loc_31307D
call    sub_3140F3
jmp     short loc_31308C
    
```

```

loc_31307D:
; CODE XREF: sub_312FD8+49
; sub_312FD8+49
call    sub_3140F3
test    eax, eax
jg      short loc_31308C
call    sub_3140F3
jmp     short loc_31308C

loc_31308C:
; CODE XREF: sub_312FD8+49
; sub_312FD8+49
call    sub_3140F3
test    eax, eax
jg      short loc_31309A
call    sub_3140F3
jmp     short loc_3130A4
    
```

Heap Spray in Action



← 0x00000000 – Start of memory

← 0x08048000 – .text Segment in ELF

← 0x09104000 – Top of heap

```
filler = "AAAAAAAAAAAAAAAA";
for(i = 0; i < 3000; i++)
```

```
{
    temp = malloc(1000000);
    memcpy(temp, filler, 1000000);
}
```

← 0xbbe09e00 – bottom of heap

← 0xbffff000 – Top of stack

← 0xFFFFFFFF – End of memory

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
```

```
push    esi
push    edi
mov     [ebp+arg_0], eax
call    sub_31486A
test    eax, eax
```

```
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 1D0h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
```

```
test    eax, eax
jz      short loc_31306E
[ebp+arg_0], esi
short loc_31306F
```

```
loc_313066: ; CODE XREF: sub_312FD8
; sub_312FD8+56
```

```
push    esi
call    sub_314118
```

```
loc_31306E: ; CODE XREF: sub_312FD8+58
; sub_312FD8+49
```

```
call    sub_3140F3
```

```
test    eax, eax
jz      short loc_31306E
call    sub_3140F3
```

```
jmp     short loc_31308C
```

```
loc_31307F: ; CODE XREF: sub_312FD8
```

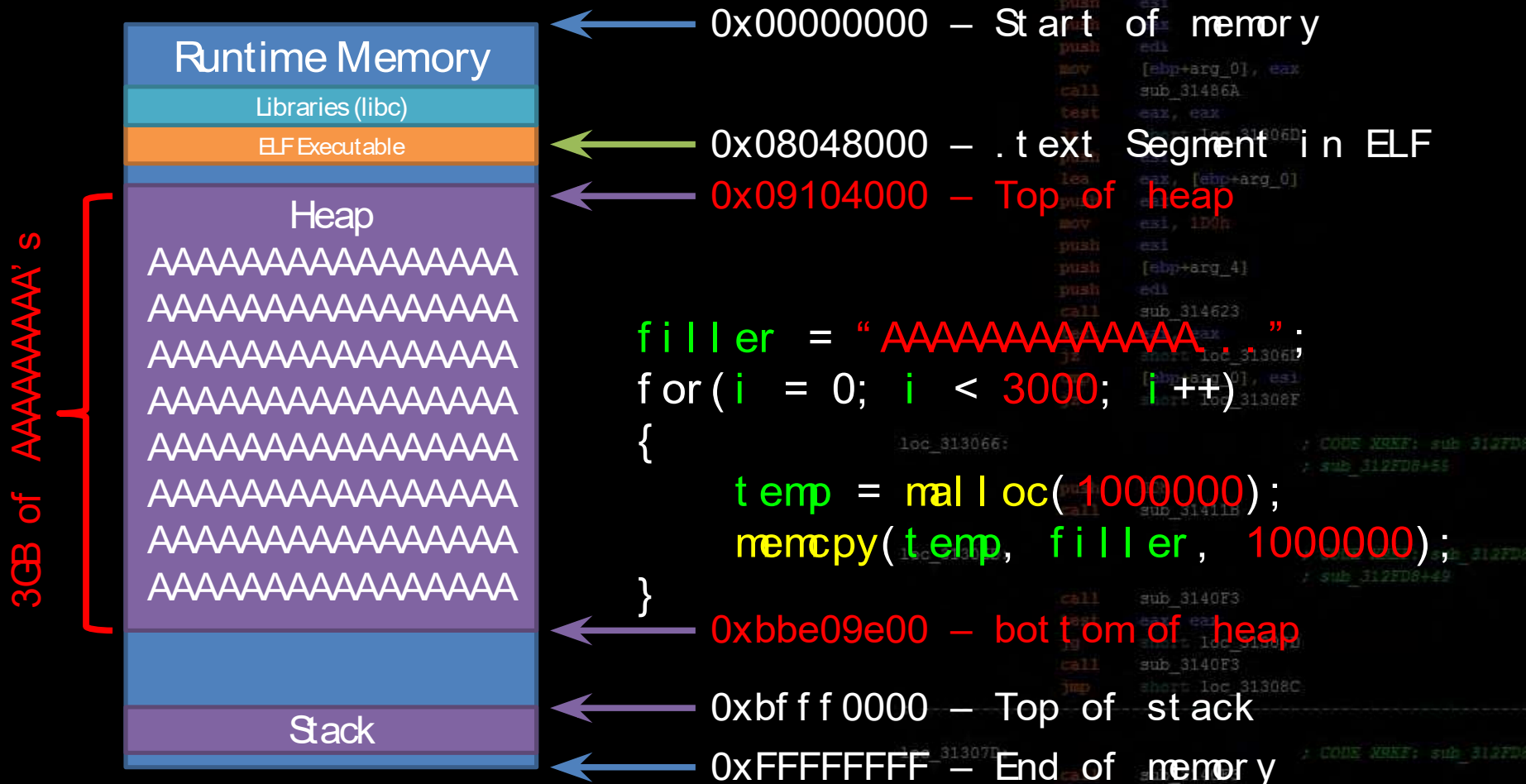
```
call    sub_3140F3
```

```
test    eax, eax
jz      short loc_31306E
```

```
loc_31308C: ; CODE XREF: sub_312FD8+49
```

```
mov     [ebp+var_41], eax
```

Heap Spray in Action



Heap Spraying in the Wild

- Generally found in browser **exploits**, rare in CTF and wargames but still something you should be aware of
- Usually **heap sprays** are done in something like javascript placed on a malicious html page

```
memory = new Array();  
for (i = 0; i < 0x100; i ++)  
    memory[i] = ROPNOP + ROP;
```

```
push    edi  
call    sub_314623  
test    eax, eax  
jz      short loc_31306D  
cmp     [ebp+arg_0], ebx  
jnz     short loc_313066  
mov     eax, [ebp+var_70]  
cmp     eax, [ebp+var_84]  
jb      short loc_313066  
sub     eax, [ebp+var_84]  
push    esi
```

```
push    esi  
push    eax  
mov     [ebp+arg_1], eax  
call    sub_31486A  
test    eax, eax  
jz      short loc_31306D  
push    esi  
lea     eax, [ebp+arg_0]  
push    eax  
mov     esi, 1D0h  
push    esi  
push    [ebp+arg_4]  
push    edi  
call    sub_314623  
mov     [ebp+arg_1], esi  
cmp     [ebp+arg_0], esi  
jz      short loc_31306F
```

```
loc_313066:                                     ; CODE XREF: sub_312FD8  
                                                ; sub_312FD8+56
```

```
push    0Dh  
call    sub_31411B
```

```
loc_31306D:                                     ; CODE XREF: sub_312FD8  
                                                ; sub_312FD8+49
```

```
call    sub_3140F3  
test    eax, eax  
jg      short loc_31307D  
call    sub_3140F3  
jmp     short loc_31308C
```

```
loc_31307D:                                     ; CODE XREF: sub_312FD8  
call    sub_3140F3
```

```
mov     [ebp+var_41], eax
```

Heap Spraying on 32bit

- On 32bit systems your address space is at maximum 4GB (2^{32} bytes)
- Spray 3GB of A's onto the heap?
 - +75% chance of 0x23456789 being a valid pointer!
 - Note: It's unlikely you would ever need to spray 3GB of anything as heap locations can be somewhat predictable, even with ASLR

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
```

```
push    esi
push    eax
push    esi
call    sub_31486A
test    eax, eax
jz      short loc_31306D
push    esi
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 1D0h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], esi
```

```
loc_313066:                                ; CODE XREF: sub_312FD8
; sub_312FD8+56
```

```
call    sub_31411B
```

```
loc_313069:                                ; CODE XREF: sub_312FD8
; sub_312FD8+49
```

```
call    sub_3140F3
test    eax, eax
jz      short loc_31307D
call    sub_3140F3
jmp     short loc_31308C
```

```
loc_31307D:                                ; CODE XREF: sub_312FD8
call    sub_3140F3
```

```
mov     [ebp+var_41], eax
```

Heap Spraying on 64bit

- On 64bit heap spraying can't really be used to bypass **ASLR**
 - Good luck spraying anywhere near 2^{64} bytes (spoiler: that's ~18446744 terabytes)
- Targeted sprays are still useful in scenarios that you have a partial heap ptr overwrite or need to do some heap grooming

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
```

```
push    esi
push    eax
push    esi
call    sub_31486A
test    eax, eax
jz      short loc_31306D
push    esi
lea     eax, [ebp+arg_0]
push    eax
push    [ebp+arg_4]
jnz     edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31306F
```

```
loc_31306F: ; CODE XREF: sub_312FD8+55
push    esi
call    sub_31411B
loc_31307D: ; CODE XREF: sub_312FD8+59
call    sub_3140F3
test    eax, eax
jg      short loc_31307D
call    sub_3140F3
jmp     short loc_31308C
```

```
loc_31307D: ; CODE XREF: sub_312FD8+59
call    sub_3140F3
```

```
mov     [ebp+var_41], eax
```

Heap Spray Payloads

- Pretty common to spray some critical value for your **exploit**, fake objects, or **ROP** chains

```
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], ebx
jnz     short loc_313066
mov     eax, [ebp+var_70]
cmp     eax, [ebp+var_84]
jb      short loc_313066
sub     eax, [ebp+var_84]
push    esi
```

```
push    esi
push    eax
call    sub_31486A
test    eax, eax
jz      short loc_31306D
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 1D0h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
test    eax, eax
jz      short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31306F
```

```
loc_313066:                                     ; CODE XREF: sub_312FD8
                                              ; sub_312FD8+56
```

```
push    0Dh
call    sub_31411B
```

```
loc_31306D:                                     ; CODE XREF: sub_312FD8
                                              ; sub_312FD8+49
```

```
call    sub_3140F3
test    eax, eax
jg      short loc_31307D
call    sub_3140F3
jmp     short loc_31308C
```

```
loc_31307D:                                     ; CODE XREF: sub_312FD8
```

```
call    sub_3140F3
```

```
mov     [ebp+var_4], eax
```

End of Lecture 5